

Ciência da Computação 8NA

Oráculo dos Direitos Trabalhistas

Sprints 01 a 09: do planejamento à entrega

Apresentado para Pryscilla de Barros Gonçalves
Preparado pela equipe Kaleido Void Studios

Índice

01 1. Identificação da Equipe

01 2. Escolha do Tema

01 3. Definição do Problema

02 4. Objetivos

03 5. Público-alvo

03 6. Requisitos Funcionais

04 7. Requisitos Não Funcionais

05 8. Casos de Uso

05 9. Product Backlog Inicial

07 10. Cronograma Inicial

07 11. Repositório GitHub

08 12. Tecnologias Definidas

09 13. Andamento do Desenvolvimento

11 14. Identidade Visual

12 Sprint 02 — Arquitetura, modelagem e protótipos

19 Sprint 03 — Base funcional: login, perfis e CRUD

24 Sprint 04 — Módulo 1: gestão da base de conhecimento

34 Sprint 05 — Módulo 2: acompanhamento de temas

40 Sprint 06 — Módulo 3, integração e melhorias de interface

47 Sprint 07 — Dashboard, relatórios, filtros, exportação e histórico

52 Sprint 08 — Consolidação: permissões, usabilidade e regras de negócio

56 Sprint 09 — Verificação completa e documentação

60 Paralelismo e concorrência no Oráculo

06 Avaliação das respostas da IA

65 Registro de decisões e mudanças de escopo

69 Histórico de alterações do relatório

1. Identificação da Equipe

- **Curso:** Ciência da Computação
- **Turma:** 8NA
- **Disciplinas:** Fábrica de Software, Tópicos Avançados e DevOps Tools
- **Projeto:** Oráculo dos Direitos Trabalhistas
- **Repositório:** https://github.com/PedroHTLeal/oraculo_direitos_trabalhistas_8na
- **Integrantes:**
- Pedro Henrique Tenório Leal - 01591056 - 8NA
- Gerson Martinho Mesquita Neto - 01604283 - 8NA

2. Escolha do Tema

A proposta é desenvolver uma plataforma capaz de receber dúvidas dos usuários em linguagem natural e apresentar respostas de forma mais simples e acessível, utilizando como base conteúdos de fontes confiáveis, inicialmente tendo como referência a Cartilha de Direitos e Obrigações no Trabalho do Tribunal Regional do Trabalho da 4ª Região (TRT4).

O sistema utilizará uma abordagem baseada em RAG (Retrieval-Augmented Generation), buscando informações relevantes na base de conhecimento antes da geração da resposta.

O projeto está inserido na área de Tecnologia da Informação aplicada ao acesso à informação sobre direitos e obrigações trabalhistas, utilizando recursos de Inteligência Artificial para facilitar a consulta e a compreensão de conteúdos relacionados ao tema.

Dessa forma, o mesmo projeto será utilizado pelas três disciplinas, com diferentes enfoques dentro do processo de desenvolvimento:

- **Fábrica de Software:** organização e gestão do desenvolvimento;
- **Tópicos Avançados:** Inteligência Artificial, RAG, LLM e avaliação das respostas;
- **DevOps Tools:** versionamento, integração, testes, automação e entrega do software.

3. Definição do Problema

O acesso a informações sobre direitos e obrigações trabalhistas pode ser dificultado pela utilização de termos jurídicos, pela quantidade de informações disponíveis e pela dificuldade de localizar e interpretar conteúdos confiáveis. Esse problema pode afetar especialmente pessoas que possuem pouca

familiaridade com informações jurídicas ou com ferramentas digitais, dificultando a compreensão de situações relacionadas ao trabalho.

Entre os públicos que podem encontrar essas dificuldades estão trabalhadores informais ou de aplicativos, trabalhadores domésticos, pessoas com menor escolaridade e jovens que estão ingressando no mercado de trabalho, incluindo estudantes e estagiários.

Diante disso, surge a necessidade de uma solução que facilite a busca e a compreensão dessas informações, utilizando uma linguagem mais acessível e mantendo a indicação da fonte utilizada.

O problema central do projeto pode ser definido como:

> Como facilitar o acesso e a compreensão de informações sobre direitos e obrigações trabalhistas por meio de uma plataforma digital que permita ao usuário realizar perguntas em linguagem natural e obter respostas baseadas em fontes confiáveis?

4. **Objetivos**

4.1. Objetivo geral

Facilitar o acesso à informação sobre direitos e obrigações trabalhistas por meio de uma plataforma com Inteligência Artificial capaz de responder dúvidas em linguagem acessível, utilizando fontes confiáveis.

4.2. Objetivos específicos

- Permitir que o usuário realize perguntas utilizando linguagem natural.
- Facilitar a localização de informações relacionadas aos direitos e obrigações trabalhistas.
- Utilizar uma base de conhecimento confiável para fundamentar as respostas.
- Aplicar uma abordagem de RAG para recuperar informações relevantes antes da geração da resposta.
- Apresentar as respostas em linguagem mais simples e compreensível.
- Informar a fonte utilizada para cada resposta.
- Permitir que o usuário consulte o conteúdo original utilizado como referência.
- Evitar que a Inteligência Artificial forneça respostas sem informações suficientes na base de conhecimento.
- Permitir que o usuário solicite conteúdos complementares por e-mail.
- Permitir que o usuário escolha temas para acompanhar e receber atualizações.

4.3. Resultados esperados

Espera-se que o sistema facilite o acesso a informações trabalhistas, reduza a dificuldade de interpretação de conteúdos jurídicos e permita que o usuário encontre informações relevantes de maneira simples e direta.

Também se espera criar uma estrutura que permita a evolução da plataforma durante as próximas Sprints, incluindo o desenvolvimento da aplicação, integração com banco de dados, Inteligência Artificial, envio de e-mails e práticas de DevOps.

5. Público-alvo

O público-alvo principal do sistema são trabalhadores que desejam obter informações sobre seus direitos e obrigações trabalhistas.

O projeto terá atenção especial para:

- trabalhadores informais e trabalhadores de aplicativos;
- trabalhadores domésticos;
- pessoas com menor familiaridade com termos jurídicos;
- pessoas com menor familiaridade com ferramentas digitais;
- jovens em seu primeiro contato com o mercado de trabalho;
- estudantes e estagiários.

A interface deverá priorizar simplicidade, clareza e facilidade de utilização, permitindo que o usuário consiga realizar uma consulta sem a necessidade de possuir conhecimentos técnicos ou jurídicos.

6. Requisitos Funcionais

ID	Requisito	Situação
RF01	O sistema deverá permitir o acesso às funcionalidades básicas sem a necessidade de criação de uma conta.	Atendido
RF02	O sistema deverá apresentar perguntas e temas sugeridos na página inicial.	Atendido
RF03	O sistema deverá permitir que o usuário digite uma pergunta livremente.	Atendido
RF04	O sistema deverá utilizar uma abordagem RAG para buscar informações relevantes na base de conhecimento.	Atendido
RF05	O sistema deverá gerar respostas utilizando as informações recuperadas da base de conhecimento.	Atendido
RF06	O sistema deverá apresentar as respostas em linguagem acessível ao usuário.	Atendido
RF07	O sistema deverá informar a fonte utilizada para a resposta.	Atendido
RF08	O sistema deverá permitir que o usuário consulte o conteúdo original utilizado como referência.	Atendido

ID	Requisito	Situação
RF09	O sistema deverá informar quando não encontrar informações suficientes na base de conhecimento para responder à pergunta.	Atendido
RF10	O sistema deverá permitir que o usuário solicite informações complementares sobre o assunto pesquisado.	Atendido
RF11	O sistema deverá permitir o envio de conteúdos complementares por e-mail quando solicitado pelo usuário.	Atendido
RF12	O sistema deverá permitir que o usuário selecione temas para acompanhar.	Atendido
RF13	O sistema deverá permitir o envio de atualizações relacionadas aos temas selecionados pelo usuário.	Atendido
RF14	O sistema deverá possuir uma área administrativa.	Atendido

7. Requisitos Não Funcionais

ID	Requisito
RNF01	O sistema deverá ser acessível por computadores e dispositivos móveis.
RNF02	A interface deverá priorizar uma experiência simples e acessível, principalmente em dispositivos móveis.
RNF03	O sistema deverá adotar medidas relacionadas à proteção dos dados dos usuários, considerando os princípios da LGPD.
RNF04	A área administrativa deverá possuir mecanismo de autenticação.
RNF05	O sistema deverá apresentar um tempo de resposta aceitável para as consultas realizadas pelo usuário.
RNF06	O sistema deverá possuir mecanismos de registro de erros para facilitar a identificação de problemas.
RNF07	O sistema deverá permanecer disponível para utilização contínua, dentro das condições de infraestrutura definidas durante o desenvolvimento.
RNF08	As respostas geradas pela Inteligência Artificial deverão buscar consistência e aderência às informações existentes na base de conhecimento.
RNF09	A solução deverá considerar o custo operacional relacionado ao uso dos recursos de Inteligência Artificial e das integrações externas.
RNF10	O código-fonte deverá utilizar um sistema de controle de versão para registrar e acompanhar as alterações realizadas pela equipe.
RNF11	O projeto deverá possuir organização adequada do repositório para facilitar o trabalho colaborativo entre os integrantes.
RNF12	O processo de desenvolvimento deverá contemplar testes e validações antes da disponibilização de novas versões do sistema.
RNF13	Sempre que tecnicamente viável, etapas de integração, testes e disponibilização deverão ser automatizadas.
RNF14	O sistema deverá possuir um processo organizado de implantação e atualização de suas versões.

Observação: as ferramentas que atendem a esses requisitos estão na seção 12: testes automatizados (pytest e ESLint), integração contínua no GitHub Actions (RNF12, RNF13) e publicação automática na Vercel a cada envio ao GitHub (RNF07, RNF14).

8. Casos de Uso

8.1. Atores

O sistema terá inicialmente dois atores principais:

- **Usuário:** pessoa que acessa a plataforma para consultar informações sobre direitos e obrigações trabalhistas.
- **Administrador:** responsável pela manutenção da base de conhecimento e pela auditoria das informações geradas pelo sistema.

8.2. Casos de uso do Usuário

ID	Caso de uso	Descrição
UC01	Realizar pergunta	O usuário informa uma dúvida sobre direitos ou obrigações trabalhistas.
UC02	Receber resposta	O sistema processa a pergunta, consulta a base de conhecimento e apresenta uma resposta acessível.
UC03	Consultar fonte	O usuário pode visualizar a fonte utilizada para fundamentar a resposta.
UC04	Solicitar conteúdo complementar	O usuário solicita informações adicionais relacionadas ao assunto consultado.
UC05	Receber conteúdo por e-mail	O usuário informa nome e e-mail para receber conteúdos complementares.
UC06	Acompanhar tema	O usuário seleciona um tema para receber futuras atualizações relacionadas a ele.

8.3. Casos de uso do Administrador

ID	Caso de uso	Descrição
UC07	Atualizar base de conhecimento	O administrador adiciona, atualiza ou remove conteúdos utilizados pelo sistema.
UC08	Auditar perguntas	O administrador consulta os registros de perguntas para acompanhar o funcionamento do sistema, sem expor dados sensíveis dos usuários.

9. Product Backlog Inicial

O Product Backlog representa as principais atividades e funcionalidades planejadas para a evolução do projeto.

ID	Item	Descrição	Área	Situação
PB01	Levantamento de requisitos	Definir e documentar as necessidades do sistema.	Fábrica de Software	Concluído

ID	Item	Descrição	Área	Situação
PB02	Modelagem do banco de dados	Estruturar as informações que serão armazenadas pelo sistema.	Desenvolvimento	Concluído
PB03	Desenvolvimento do backend	Criar a estrutura responsável pelo processamento das requisições.	Desenvolvimento	Concluído
PB04	Desenvolvimento do frontend	Criar a interface de interação com o usuário.	Desenvolvimento	Concluído
PB05	Construção da base de conhecimento	Organizar os conteúdos utilizados pelo sistema.	Tópicos Avançados	Concluído
PB06	Implementação do RAG	Desenvolver o mecanismo de recuperação das informações relevantes.	Tópicos Avançados	Concluído
PB07	Integração com banco de dados	Integrar a aplicação à estrutura de armazenamento.	Desenvolvimento	Concluído
PB08	Integração com e-mail	Criar o fluxo para envio de conteúdos complementares e atualizações.	Integração	Concluído
PB09	Automação de envios	Automatizar o envio dos conteúdos complementares e das atualizações.	Integração / DevOps	Concluído (envio em segundo plano no backend)
PB10	Área administrativa	Desenvolver as funcionalidades para manutenção da base e auditoria.	Desenvolvimento	Concluído (painel por perfil)
PB11	Testes do sistema	Verificar o funcionamento das funcionalidades e integrações.	DevOps Tools	Concluído (Sprint 09)
PB12	Avaliação das respostas da IA	Verificar a consistência das respostas e sua relação com a base de conhecimento.	Tópicos Avançados	Concluído (15 de 15 perguntas aprovadas)
PB13	Integração geral	Integrar frontend, backend, banco de dados, IA e e-mail.	DevOps Tools	Concluído
PB14	Deploy	Preparar o sistema para disponibilização em ambiente de execução.	DevOps Tools	Concluído (Vercel)
PB15	Documentação	Registrar o desenvolvimento, decisões e funcionamento do projeto.	Fábrica de Software	Concluído
PB16	Controle de versões	Organizar o código e acompanhar as alterações realizadas pela equipe.	DevOps Tools	Concluído
PB17	Organização do repositório	Definir uma estrutura adequada para branches, commits e arquivos do projeto.	DevOps Tools	Concluído
PB18	Automação de testes	Estruturar a execução automatizada de testes durante o desenvolvimento.	DevOps Tools	Concluído (pytest)
PB19	Integração contínua	Automatizar a validação das alterações realizadas no projeto.	DevOps Tools	Concluído (GitHub Actions)
PB20	Deploy automatizado	Estruturar o processo de disponibilização de novas versões da aplicação.	DevOps Tools	Concluído (Vercel, a cada envio)

ID	Item	Descrição	Área	Situação
PB21	Monitoramento e logs	Registrar informações importantes para identificação de erros e acompanhamento do sistema.	DevOps Tools	Concluído (logs e histórico)

10. **Cronograma Inicial**

O desenvolvimento será realizado de forma incremental, utilizando Sprints para organizar a evolução do projeto.

Sprint	Atividades
Sprint 01	Requisitos, casos de uso, backlog e planejamento; definição da proposta de IA/RAG; organização do repositório e controle de versão
Sprint 02	Planejamento das funcionalidades; preparação da base de conhecimento; estruturação do ambiente de desenvolvimento *(adiantada: já concluída na Sprint 01)*
Sprint 03	Desenvolvimento do sistema; integração do código e testes *(iniciado: tela de perguntas já desenvolvida)*
Sprint 04	Integração das funcionalidades; implementação do RAG/IA; automação de testes e integração
Sprint 05	Integração geral; ajustes e avaliação da IA; integrações e automações
Sprint 06	Testes e preparação da entrega; avaliação das respostas; deploy, monitoramento e versão final

Observação: o cronograma é inicial e poderá ser ajustado conforme o andamento do projeto e as orientações realizadas durante as próximas Sprints.

Na execução, o projeto seguiu as Sprints 01 a 09 definidas pela disciplina; o que foi entregue em cada uma está nas páginas das Sprints, a seguir neste relatório.

11. **Repositório GitHub**

O projeto possui um repositório no GitHub destinado ao armazenamento e controle de versão dos arquivos desenvolvidos pela equipe.

Repositório: https://github.com/PedroHTLeal/oraculo_direitos_trabalhistas_8na

O GitHub é utilizado para organização do código, controle de versões, colaboração entre os integrantes e acompanhamento da evolução do projeto.

Estrutura do repositório:

```
backend/    API em Python (FastAPI), testes, scripts e dados da cartilha
frontend/   Interface web em React
supabase/   Estrutura do banco de dados (SQL)
docs/       Documentação do projeto, guia de configuração e relatórios das Sprints
```

Organização dos commits:

- o projeto usa um único branch, o `main`, para que toda a evolução fique em uma linha do tempo fácil de acompanhar;
- **cada ajuste é um commit**, com mensagem em português contendo a data, o objetivo, o que foi feito e como agrega ao projeto;
- os marcos são registrados no arquivo `CHANGELOG.md`, e a pasta `docs/sprints/` liga cada Sprint aos commits que a entregaram.

Nas primeiras Sprints foram usados branches por funcionalidade (`tela-perguntas`, `novo-visual`, `base-conhecimento` e `sprint-02`) e um Pull Request. Em 02/11/2026 eles foram consolidados no `main` sem perda de histórico, porque cada branch havia sido criado a partir do anterior.

A definição das automações de integração contínua e deploy será realizada durante as próximas Sprints.

12. Tecnologias Definidas

Camada	Tecnologia	Motivo
Backend	Python 3.12 + FastAPI	Python concentra as principais bibliotecas de IA e RAG; FastAPI é leve, rápido e gera documentação da API automaticamente.
Frontend	React + Vite	Componentes reutilizáveis e ambiente de desenvolvimento rápido, com interface responsiva para celular e computador.
Testes do backend	pytest	Ferramenta padrão de testes em Python, base para a automação de testes (PB18).
Qualidade do frontend	ESLint	Verificação automática de padrões e erros no código React.
Controle de versão	Git + GitHub	Versionamento e colaboração (RNF10, RNF11).
Banco de dados	Supabase (PostgreSQL)	Banco gerenciado e gratuito no início, com busca textual em português (ignorando acentos) e controle de acesso por tabela (RNF03).
Inteligência Artificial	Google Gemini (plano gratuito)	Gera respostas em linguagem simples usando só os trechos da cartilha, sem custo (RNF09).
E-mail	Gmail (SMTP) com senha de app	Gratuito, sem instalar nada além do backend; a pessoa só precisa informar nome e e-mail.
Paralelismo	Threads em Python (`concurrent.futures`)	As tarefas esperam a rede (banco e Gmail); em paralelo, o envio das novidades e o dashboard ficam mais rápidos (documento).

Camada	Tecnologia	Motivo
Integração contínua	GitHub Actions	Roda os testes do backend e confere o frontend a cada envio ao GitHub, sem custo (PB19).
Publicação	Vercel (plano gratuito)	Publica o site e o backend juntos e atualiza a cada envio ao `main` (PB14, PB20).

13. Andamento do Desenvolvimento

Tela de perguntas

Primeira tela do sistema, que atende ao caso de uso UC01 (Realizar pergunta):

- campo para o usuário digitar a pergunta livremente, com limite de 500 caracteres (RF03);
- temas e perguntas sugeridos na página inicial — Férias, 13º salário, Jornada de trabalho, Rescisão, Estágio e Trabalho doméstico (RF02);
- acesso sem criação de conta (RF01);
- cartão de resposta preparado para exibir a fonte e o link para o conteúdo original (RF07, RF08);
- aviso destacado quando não há informação suficiente na base de conhecimento (RF09);
- layout adaptado para celular e computador (RNF01, RNF02);
- aviso de que as respostas têm caráter informativo.

Na primeira versão, a API respondia que ainda não havia informações suficientes, já no formato final (resposta, fontes e indicação se encontrou). Com isso, a tela não precisou mudar quando a IA foi conectada (ver "Base de conhecimento, IA e e-mail" abaixo).

Base de conhecimento, IA e e-mail

Base de conhecimento (PB05): um script extrai automaticamente as 25 páginas da Cartilha de Direitos e Obrigações no Trabalho do TRT4 e as divide em 217 trechos, um por subtítulo, cada um com o tema, a seção e o link da página original. Os trechos ficam salvos no repositório (`backend/dados/cartilha.json`) e são carregados no Supabase.

Banco de dados (PB02, PB07): o Supabase guarda três tabelas:

Tabela	Conteúdo
`trechos`	Os trechos da cartilha, com um índice de busca em português que ignora acentos
`perguntas`	As perguntas feitas e as respostas geradas, para auditoria (UC08), sem dados pessoais
`solicitacoes_conteudo`	Os pedidos de conteúdo por e-mail: nome, e-mail, consentimento, situação do envio e materiais enviados
`sinonimos`	Palavras do dia a dia e seus equivalentes na cartilha (ex.: "demitido" → "dispensa", "patrão" → "empregador"), editáveis pela equipe no painel, na tela Sinônimos da busca (UC07)

O acesso às tabelas é bloqueado para o público; só o backend, com a chave de serviço, lê e grava os dados (RNF03).

Como a resposta é gerada (RAG, PB06):

1. A pergunta é usada para buscar no Supabase os 6 trechos mais relevantes. A busca ignora acentos, acrescenta os sinônimos das palavras do dia a dia, dá mais peso a palavras no tema e no subtítulo e às palavras raras na cartilha (por exemplo, "diarista" pesa mais que "carteira") e limita a 3 trechos por tema, para a IA ver mais de um assunto quando a pergunta envolve vários (ex.: "estagiário tem direito a férias?").
2. A IA recebe somente esses trechos e escreve uma resposta curta e simples (RF05, RF06), indicando quais trechos usou.
3. O sistema confere se os trechos citados vieram mesmo da busca e os mostra como fonte, com o texto original e o link para a cartilha (RF07, RF08).
4. Se nenhum trecho for encontrado, ou se a IA concluir que eles não respondem à pergunta, a tela informa que não há informação suficiente (RF09).

Em computadores, a resposta aparece no painel azul, no lugar da órbita de temas; no celular, logo abaixo do campo de pergunta.

Saber mais por e-mail (RF10, RF11, UC04, UC05, PB08):

1. Abaixo da resposta, o convite "Quer saber mais sobre o assunto?" com o botão "Sim" abre uma janela que pede o nome, o e-mail e o consentimento para o uso desses dados, conforme a LGPD (RNF03). Não é preciso ter conta.
2. O backend registra o pedido e escolhe os sites e vídeos do assunto da pergunta, cadastrados e conferidos pela equipe na tabela `materiais_complementares`. Quando o assunto ainda não tem links cadastrados, vão a página da cartilha do TRT4 sobre o tema, o texto oficial da lei e uma busca pronta no YouTube.
3. O backend envia pelo Gmail um e-mail com a resposta, os 2 sites e os 2 vídeos. A tela acompanha o envio e confirma quando ele termina. Cada e-mail pode fazer até 5 pedidos por hora, para evitar abuso.

Registro de erros (RNF06, PB21): falhas na IA, no banco ou no envio do e-mail são registradas no log do servidor, e um pedido que falha fica marcado como "erro" no banco, sem ficar pendente para sempre.

Situação: a cartilha já está carregada no Supabase e a busca foi validada com o banco real, com perguntas como "Quantos dias de ferias eu tenho?" (sem acento), "O que recebo se for demitido sem justa causa?", "Empregada doméstica tem FGTS?" e "Fui humilhado pelo meu chefe". Nesse teste foi encontrado e corrigido um erro na função de busca, que impedia palavras como "estágio" de serem encontradas. A geração da resposta pela IA e o envio por e-mail também foram testados no sistema real (casos CT25 a CT27 da Sprint 09), e a qualidade das respostas foi avaliada com perguntas de referência (PB12).

Testes automatizados do backend: 88 testes, cobrindo a busca e a geração da resposta, as fontes exibidas, o caso sem informação, o login e os perfis, o CRUD da base, o acompanhamento de temas, o

pedido por e-mail (consentimento, e-mail inválido, limite por hora, falha no envio), a escolha dos sites e vídeos, o dashboard, os relatórios e o histórico. Eles rodam sozinhos no GitHub Actions a cada envio.

Novo visual

A tela de perguntas recebeu a identidade visual descrita na seção 14. Em computadores, a tela é dividida em duas partes: à esquerda, o conteúdo (pergunta, resposta e temas); à direita, um painel azul com os temas girando em órbita ao redor de uma balança da justiça e uma frase sobre o propósito do sistema. No celular, o painel azul é ocultado para priorizar o conteúdo (RNF02). Os temas sugeridos passaram a ser exibidos como cartões.

14. **Identidade Visual**

Elemento	Definição
Logotipo	Símbolo circular em azul-escuro com um ponto dourado, seguido do nome "Oráculo" e do complemento "Direitos Trabalhistas"
Fonte	Plus Jakarta Sans (pesos 400 a 800)
Azul-escuro `#030C2C`	Cor principal: botões, tema selecionado, painel lateral e indicador de foco para navegação por teclado
Dourado `#E8B20F`	Cor de destaque: núcleo da órbita, ponto do logotipo, detalhes do tema selecionado e botão "Perguntar" ao passar o mouse
Texto `#1C1C1C` / `#5C5C5C`	Texto principal e texto de apoio
Bordas `#E2E2E2`	Cartões e campos, com cantos arredondados de 14px
Símbolo	Balança da justiça, representando direitos e equilíbrio nas relações de trabalho

Princípios aplicados: muito espaço em branco, títulos fortes, poucos elementos por tela, áreas de toque de no mínimo 44px para uso no celular e animação desativada automaticamente para quem configura o sistema para reduzir movimento (acessibilidade).

Sprint 02 — Arquitetura, modelagem e protótipos

Situação: Concluída

O que a Sprint pediu

Transformar o planejamento em estrutura técnica: arquitetura do sistema, diagrama de classes, modelo entidade-relacionamento (MER), modelo relacional, protótipos das telas principais, banco de dados criado e projeto organizado no GitHub.

O que foi entregue

Item pedido	Entrega	Situação	Onde ver
Arquitetura do sistema	Frontend React → API FastAPI → Supabase (PostgreSQL), Google Gemini e envio de e-mails pelo Gmail	Concluída	Diagrama de arquitetura
Diagrama de classes	Classes de serviço e rotas do backend	Concluído	Diagrama de classes
MER	Entidades: tema, trecho, pergunta, usuário, sessão, inscrição, atualização, envio, solicitação, material, histórico	Concluído	MER
Modelo relacional	14 tabelas com chaves primárias e estrangeiras	Concluído e implementado no banco	Modelo relacional
Protótipo das telas	Telas funcionais no próprio sistema (consulta, resposta, Saiba mais)	Concluído	Identidade visual, seção 14
Banco de dados criado	Supabase com a cartilha do TRT4 carregada (25 temas, 217 trechos)	Concluído	supabase/
Projeto no GitHub	Estrutura `backend/`, `frontend/`, `supabase/` e `docs/`	Concluído	Repositório

Ajustes desta Sprint

Commit	O que mudou
`6824b07`	Backend em Python/FastAPI e frontend em React/Vite
`438333f`	Primeira tela: perguntas, temas sugeridos e cartão de resposta
`2392fd3`	Identidade visual inspirada no FlowFocus (layout dividido e painel com órbitas)
`6e6f5a2`	Paleta definitiva: azul-escuro `#030C2C` e dourado `#E8B20F`
`10f67db`	Extração da cartilha do TRT4 e primeira estrutura do banco no Supabase
`1c4b4a9`	Busca validada no banco real, correção da busca e tabela de sinônimos

Commit	O que mudou
`ecd1787`	Modelo relacional completo (temas, usuários, perfis, sessões, inscrições, atualizações)

Mudanças em relação ao planejado

- **Banco de dados:** foi escolhido o Supabase (PostgreSQL gerenciado e gratuito), com busca textual em português no lugar de um banco vetorial. A cartilha tem apenas 217 trechos, e a busca textual com sinônimos se mostrou suficiente e sem custo.
- **Protótipos:** em vez de um protótipo no Figma, a equipe construiu as telas direto no sistema, que já funcionam como protótipo navegável.

Documento técnico

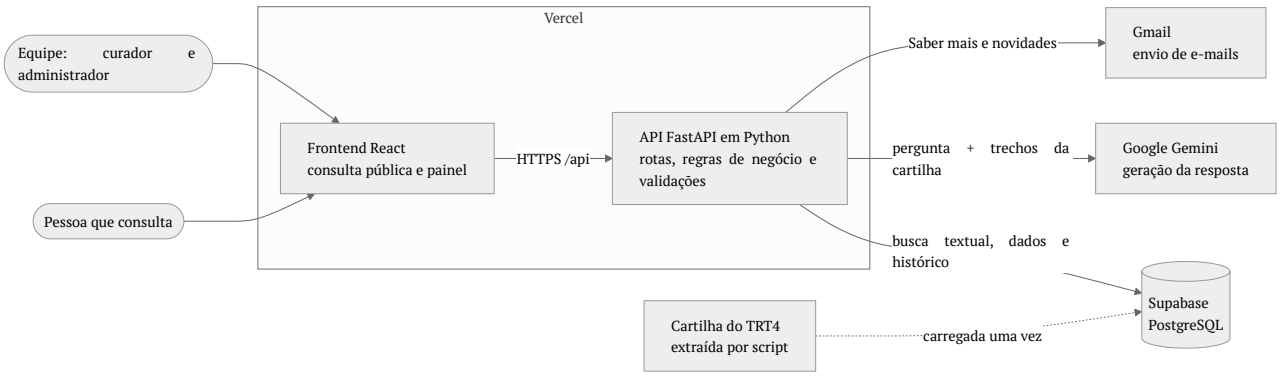
Os diagramas de arquitetura, de classes, MER e relacional, com a explicação de cada um, estão em diagramas.md.

Diagramas técnicos

Os diagramas abaixo descrevem o sistema como ele está implementado. O GitHub desenha os blocos `mermaid` automaticamente, e o relatório em PDF usa os mesmos desenhos.

1. Arquitetura do sistema

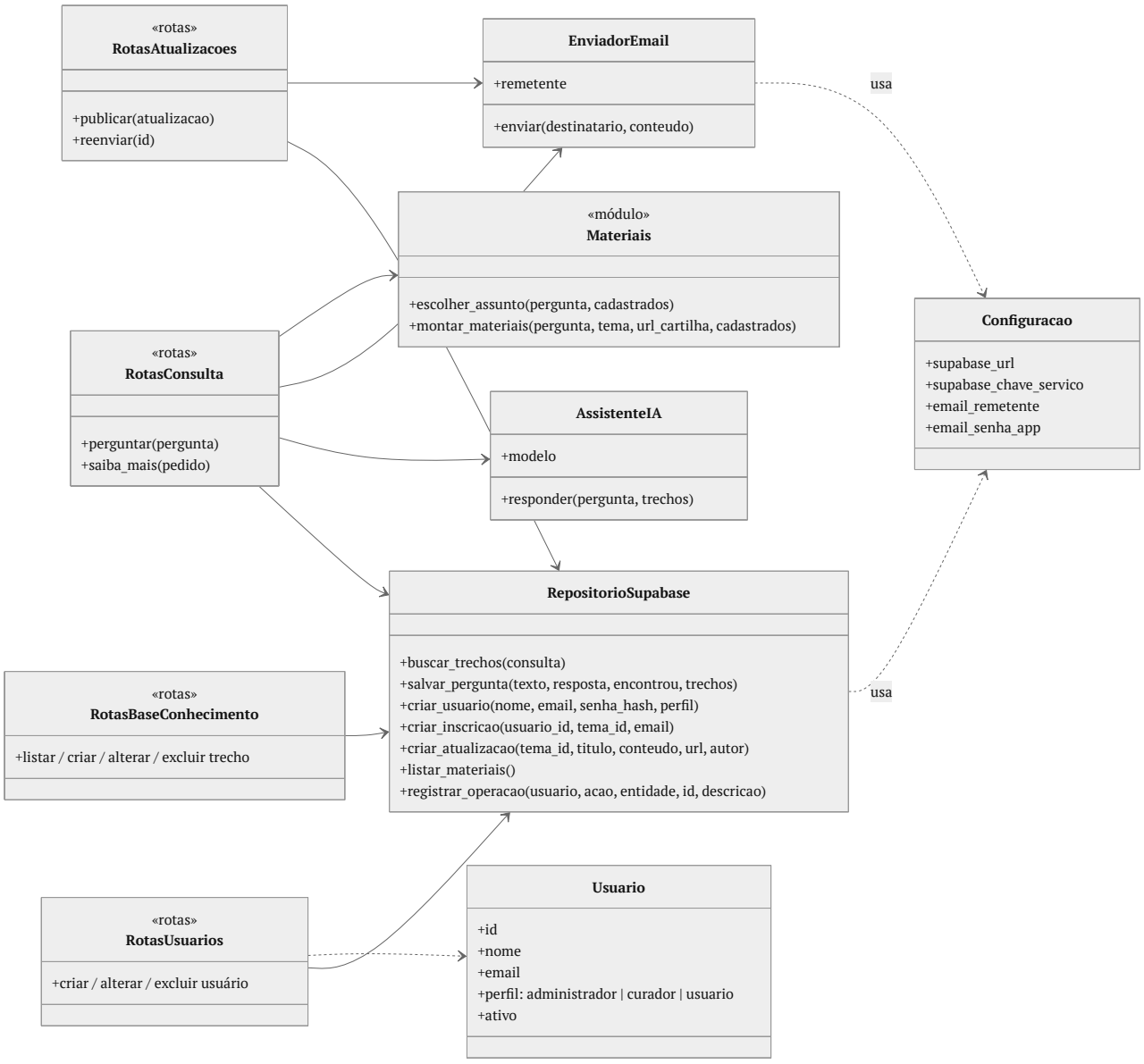
O Oráculo segue uma arquitetura em camadas: a interface em React conversa apenas com a API em Python (FastAPI), que concentra as regras de negócio e é a única parte com acesso ao banco de dados, à IA e ao envio de e-mails. As chaves de acesso ficam só no backend.



Fluxo da consulta (RAG): a pergunta chega à API, que busca no banco os trechos mais relevantes da cartilha; a IA recebe só esses trechos e escreve a resposta; a API confere as fontes citadas, grava a pergunta para auditoria e devolve a resposta à tela.

2. Diagrama de classes

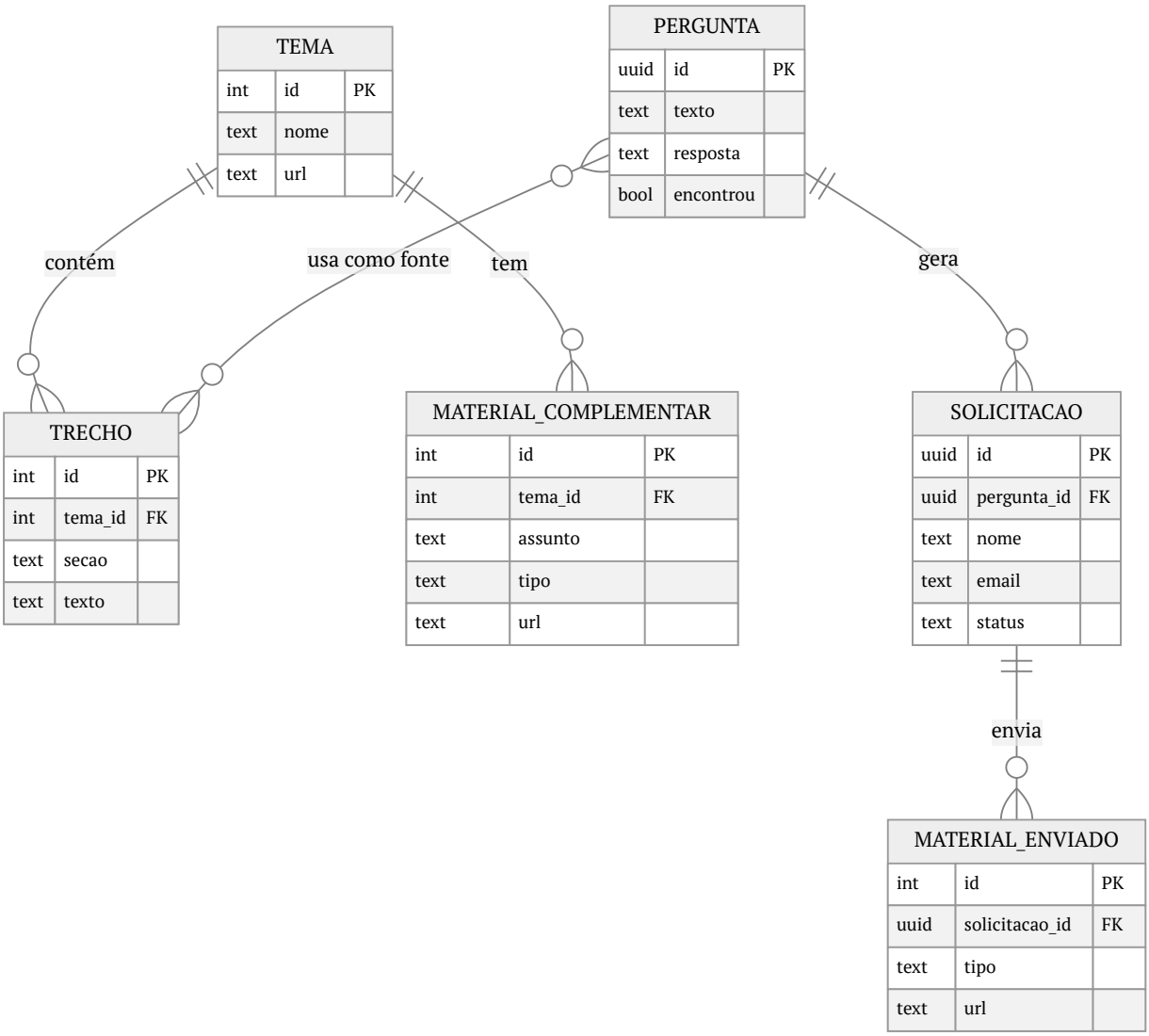
As principais classes do backend e como elas se relacionam. As rotas usam os serviços por injeção de dependência, o que permite trocar o banco, a IA e o e-mail por versões simuladas nos testes automatizados.



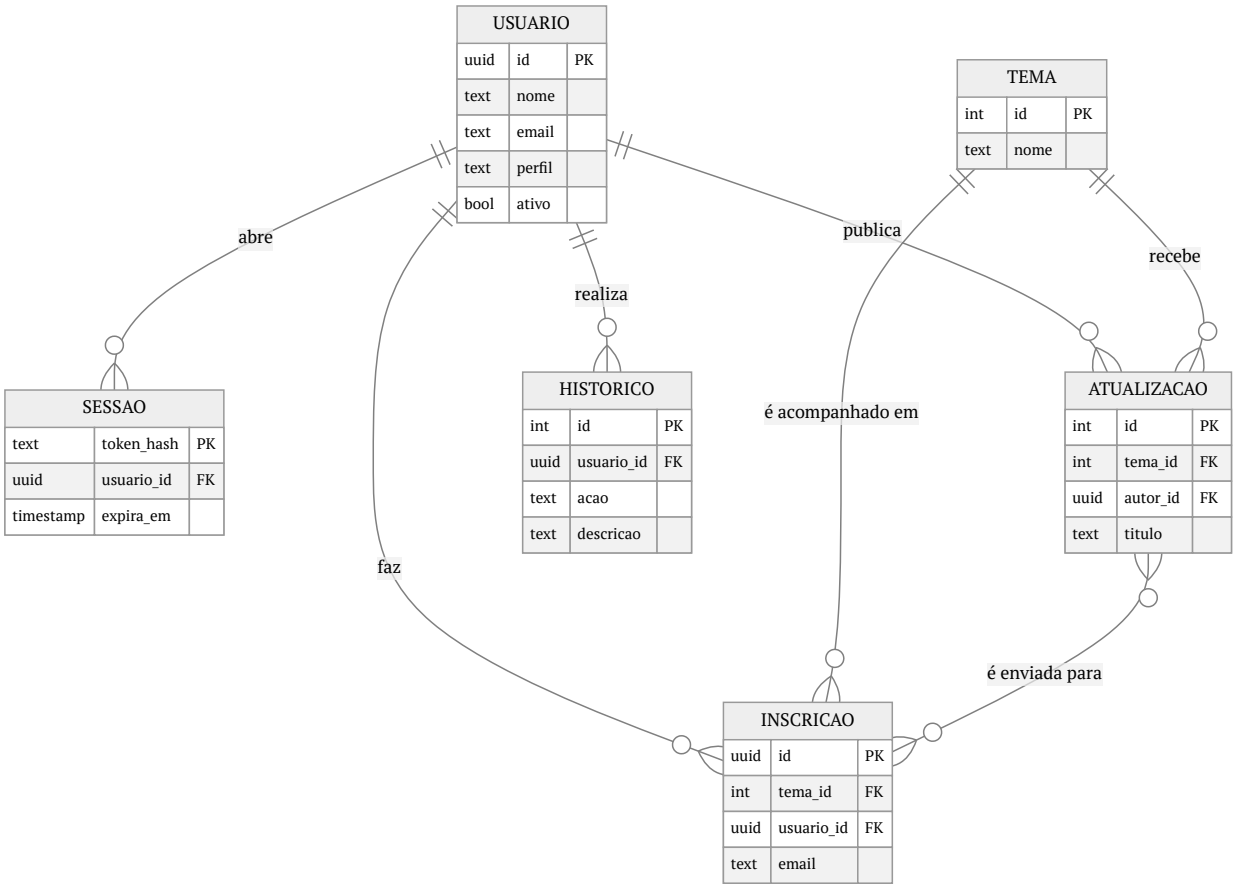
3. Modelo entidade-relacionamento (MER)

Para facilitar a leitura, o modelo está dividido em duas partes: a consulta pública e as contas da equipe e dos usuários. As entidades Tema e Usuário aparecem nas duas.

Consulta e conteúdo complementar



Contas, acompanhamento de temas e histórico



Entidade	O que representa
Tema	Uma das 25 páginas da cartilha do TRT4 (Férias, Rescisão...)
Trecho	Um subtítulo da cartilha, usado como fonte das respostas
Pergunta	Pergunta feita na consulta, com a resposta gerada (sem dados pessoais)
Solicitação	Pedido de "Saber mais": nome, e-mail, autorização e situação do envio
Material enviado	Site ou vídeo enviado em cada solicitação
Material complementar	Sites e vídeos escolhidos pela equipe para cada assunto
Usuário	Conta com perfil administrador, curador ou usuário
Sessão	Login ativo de um usuário
Inscrição	Tema que um usuário acompanha
Atualização	Novidade publicada pela equipe sobre um tema
Histórico	Registro de cada operação: quem fez, o quê e quando

4. Modelo relacional

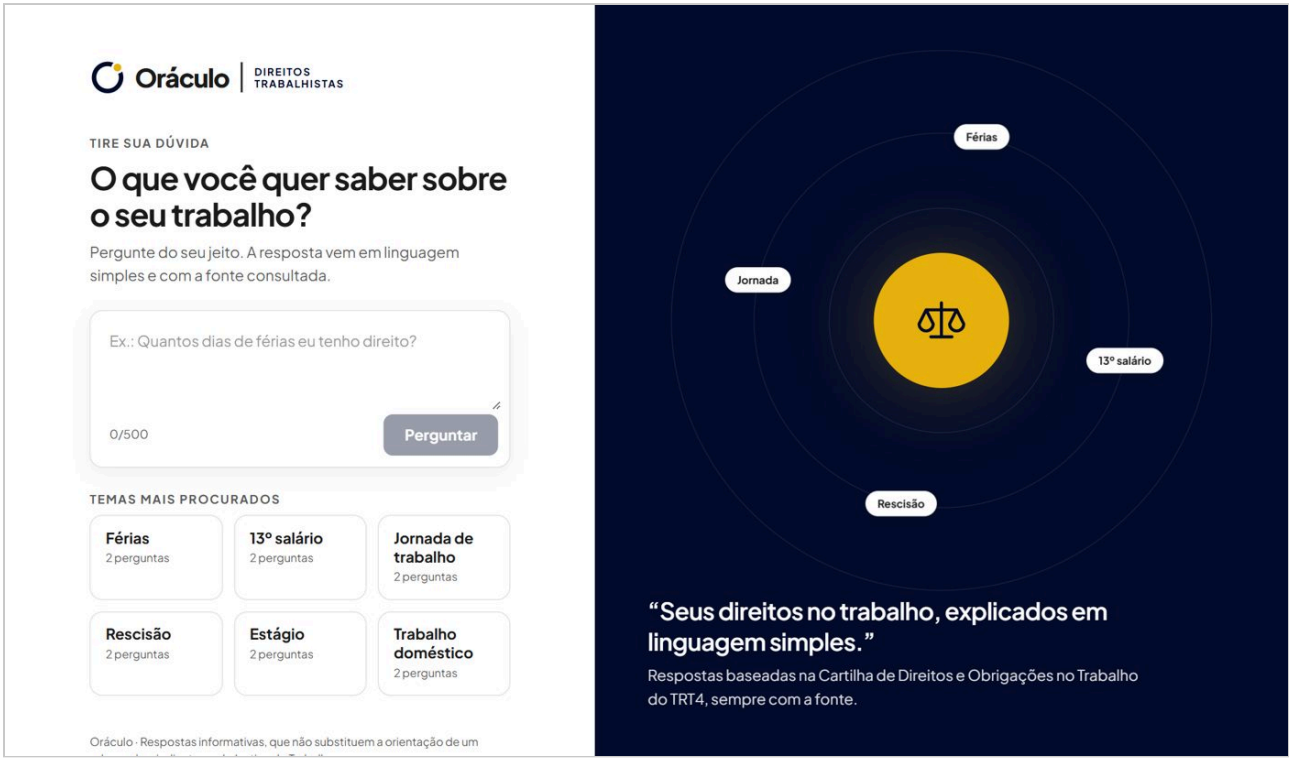
Implementado no Supabase pelos scripts `supabase/001` a `007`. **PK** = chave primária; **FK** = chave estrangeira.

Tabela	Colunas	Chaves
`temas`	id, nome, slug, url, criado_em	PK id
`trechos`	id, tema_id, secao, ordem, texto, busca, atualizado_por, atualizado_em	PK id; FK tema_id → temas; FK atualizado_por → usuarios
`sinonimos`	termo, equivalentes	PK termo
`perguntas`	id, texto, resposta, encontrou, criado_em	PK id
`pergunta_trechos`	pergunta_id, trecho_id	PK (pergunta_id, trecho_id); FK → perguntas; FK → trechos
`solicitacoes_conteudo`	id, pergunta_id, nome, email, consentimento, status, erro, criado_em, enviado_em	PK id; FK pergunta_id → perguntas
`materiais_enviados`	id, solicitacao_id, tipo, titulo, url	PK id; FK solicitacao_id → solicitacoes_conteudo
`materiais_complementares`	id, tema_id, assunto, palavras_chave, tipo, titulo, url, ordem, criado_em	PK id; FK tema_id → temas
`usuarios`	id, nome, email, senha_hash, perfil, ativo, consentimento_email, criado_em, atualizado_em	PK id; e-mail único
`sessoes`	token_hash, usuario_id, criada_em, expira_em	PK token_hash; FK usuario_id → usuarios
`inscricoes_tema`	id, tema_id, usuario_id, email, consentimento, ativa, criado_em, cancelado_em	PK id; FK tema_id → temas; FK usuario_id → usuarios; único (tema, e-mail)
`atualizacoes`	id, tema_id, autor_id, titulo, conteudo, url, publicado_em	PK id; FK tema_id → temas; FK autor_id → usuarios
`envios_atualizacao`	atualizacao_id, inscricao_id, status, enviado_em	PK (atualizacao_id, inscricao_id); FK → atualizacoes; FK → inscricoes_tema
`historico_operacoes`	id, usuario_id, usuario_nome, acao, entidade, entidade_id, descricao, criado_em	PK id; FK usuario_id → usuarios

Segurança (RNF03): todas as tabelas têm o acesso público bloqueado (RLS); só o backend, com a chave de serviço, lê e grava os dados. As senhas são guardadas apenas como hash (scrypt), e as sessões guardam só o hash do token.

5. Protótipos das telas

As telas foram construídas direto no sistema, que funciona como protótipo navegável. Os prints de cada tela estão nas Sprints 03, 04, 05, 06 e 07.



Tela de consulta

Sprint 03 — Base funcional: login, perfis e CRUD

Situação: Concluída

O que a Sprint pediu

Banco de dados conectado, login funcional, cadastro de usuários, controle de perfis, CRUD principal com persistência real e primeiro deploy local.

O que foi entregue

Item pedido	Entrega	Onde ver
Banco conectado	A API lê e grava no Supabase; a conexão é refeita sozinha se cair	`backend/app/repositorio.py`
Login funcional	Login com sessão guardada no banco (expira em 8 h); senha com hash scrypt	`rotas/autenticacao.py` · `seguranca.py`
Cadastro de usuários	Cadastro público e cadastro pelo administrador	`rotas/usuarios.py` · tela **Usuários**
Controle de perfis	**Administrador** (tudo), **Curador** (base de conhecimento e relatórios), **Usuário** (acompanha temas)	`dependencias.py` · menu do painel
CRUD principal	Base de conhecimento: cadastrar, consultar (filtro e paginação), editar e excluir trechos da cartilha	`rotas/base_conhecimento.py` · tela **Base de conhecimento**
Deploy local	Um comando instala, compila e abre o sistema em `http://localhost:8000`	`iniciar.bat` · `iniciar.sh`

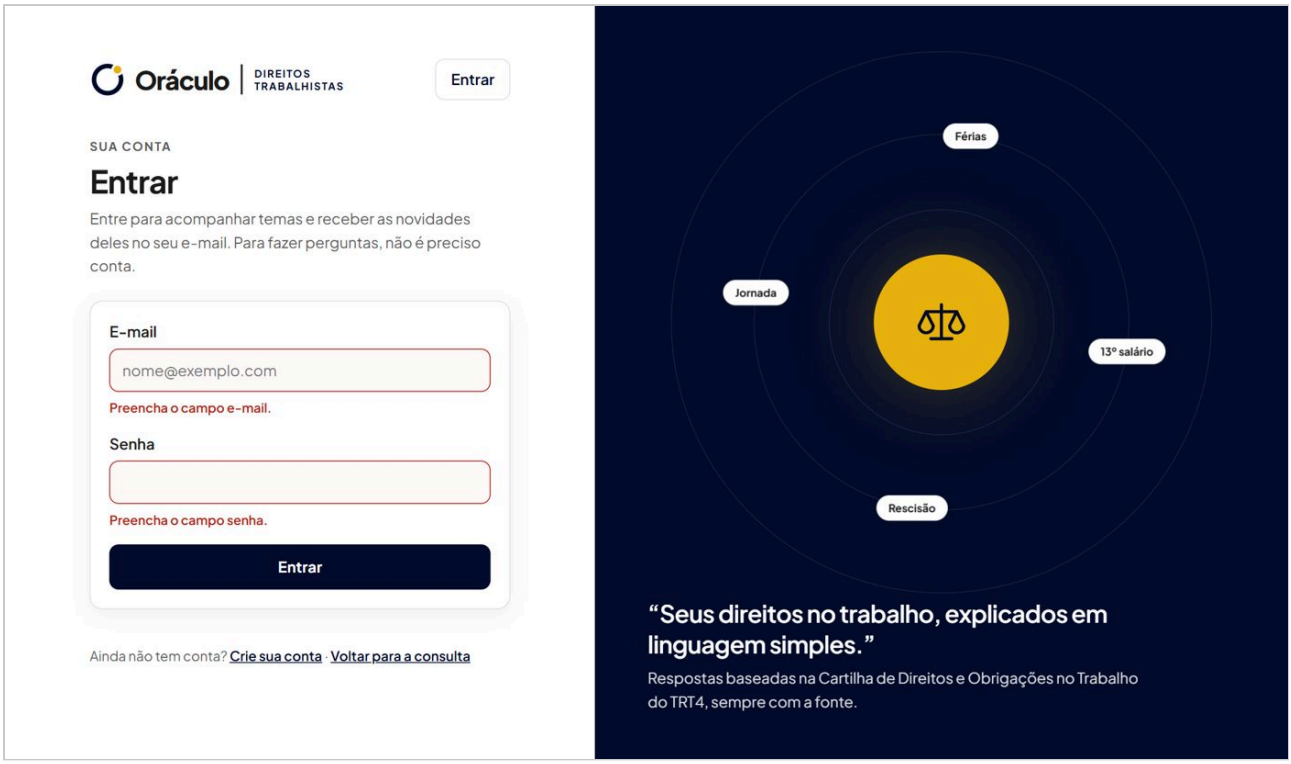
Ajustes desta Sprint

Commit	O que mudou
`5b2f043`	API organizada em módulos e erros de validação traduzidos para o português
`2d3ccea`	Login, cadastro de usuários e controle de perfis
`4f0a442`	CRUD da base de conhecimento, auditoria de perguntas e acompanhamento de temas
`e5c21bf`	Telas de login, cadastro e painel administrativo
`d9f800e`	Deploy local: o backend serve o site compilado; scripts de execução

Evidências

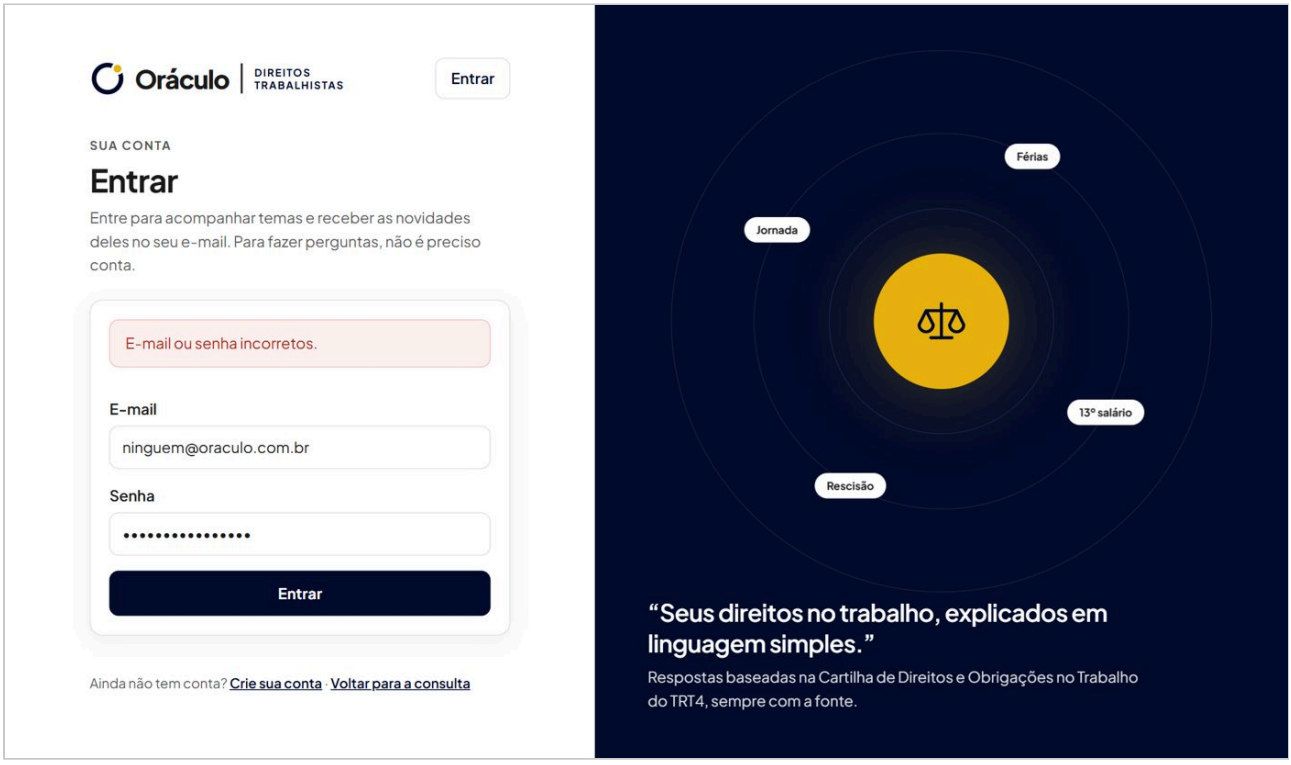
Prints gerados em 02/11/2026 pelo roteiro automatizado `capturar_evidencias.py`, com o sistema rodando localmente e o banco real no Supabase. Todos os prints estão em `docs/evidencias/`.

Login: campos obrigatórios



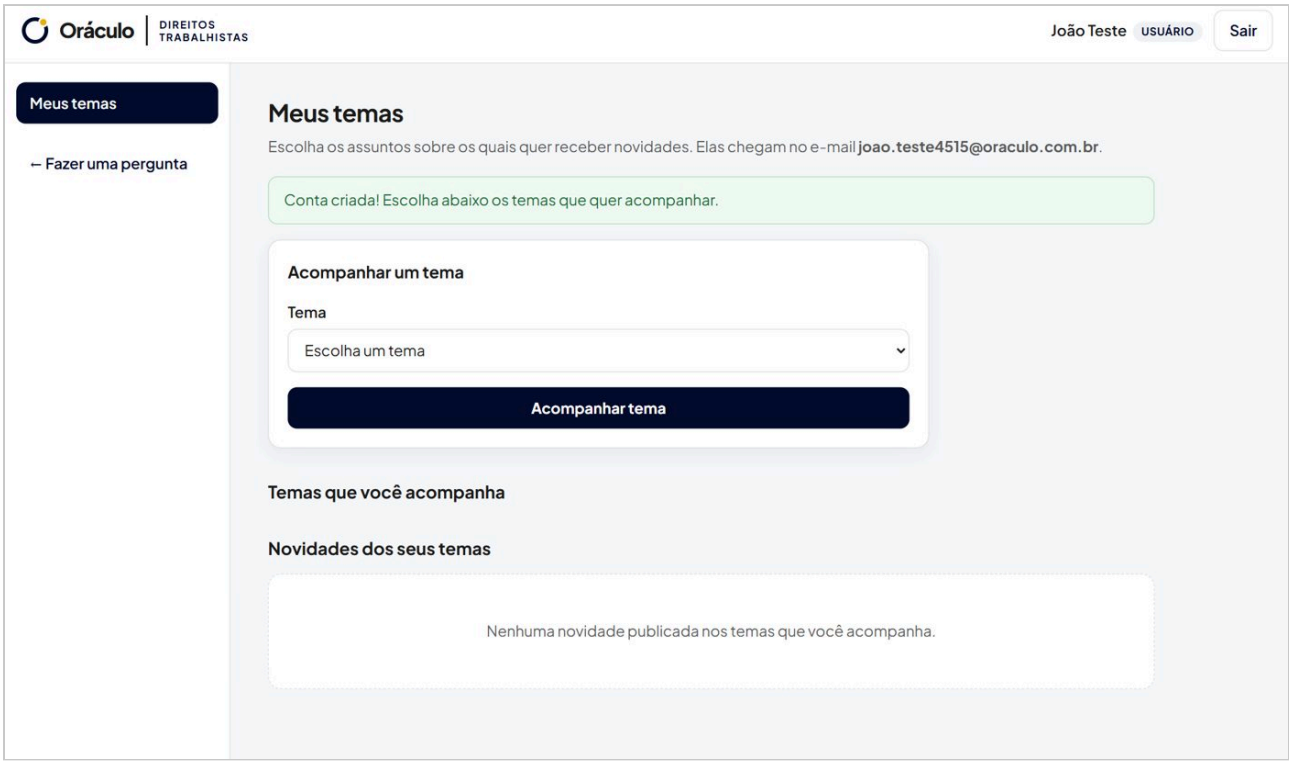
Login: campos obrigatórios

Login: e-mail ou senha incorretos



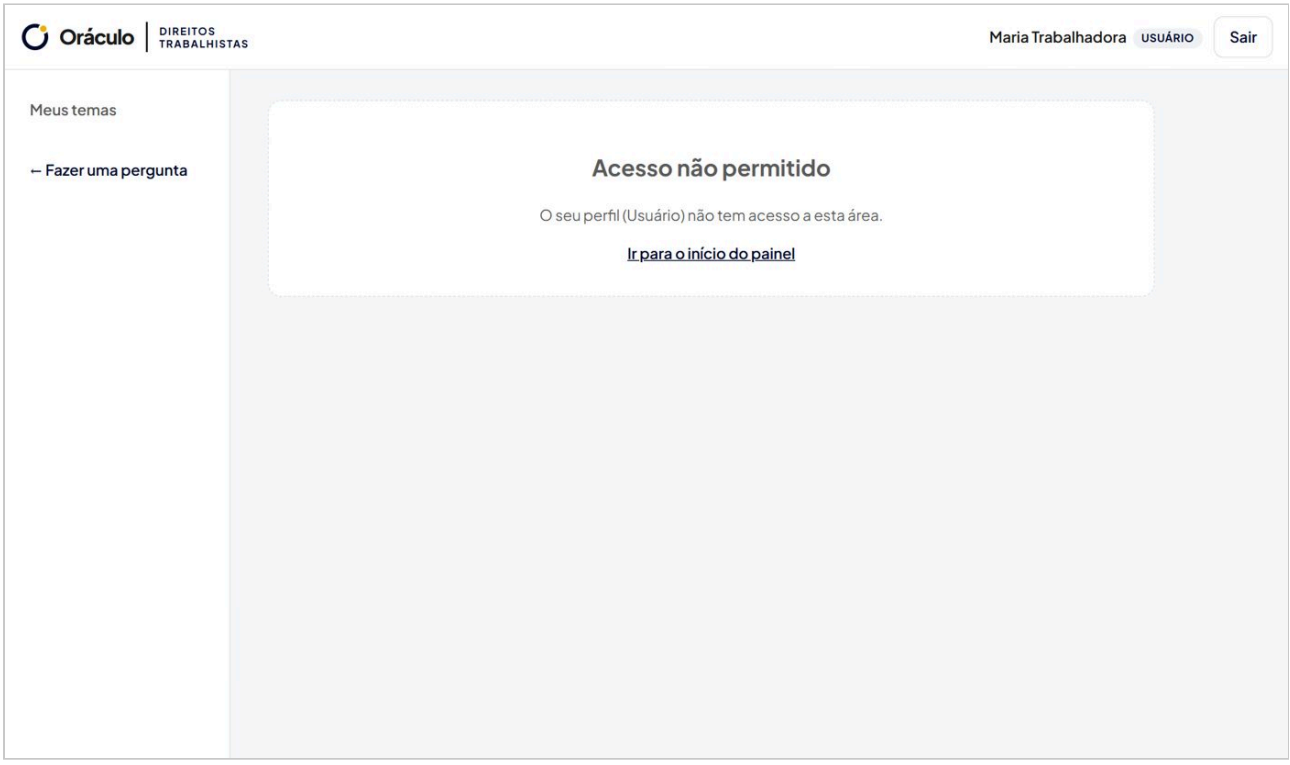
Login: e-mail ou senha incorretos

Cadastro concluído (conta gravada no banco)



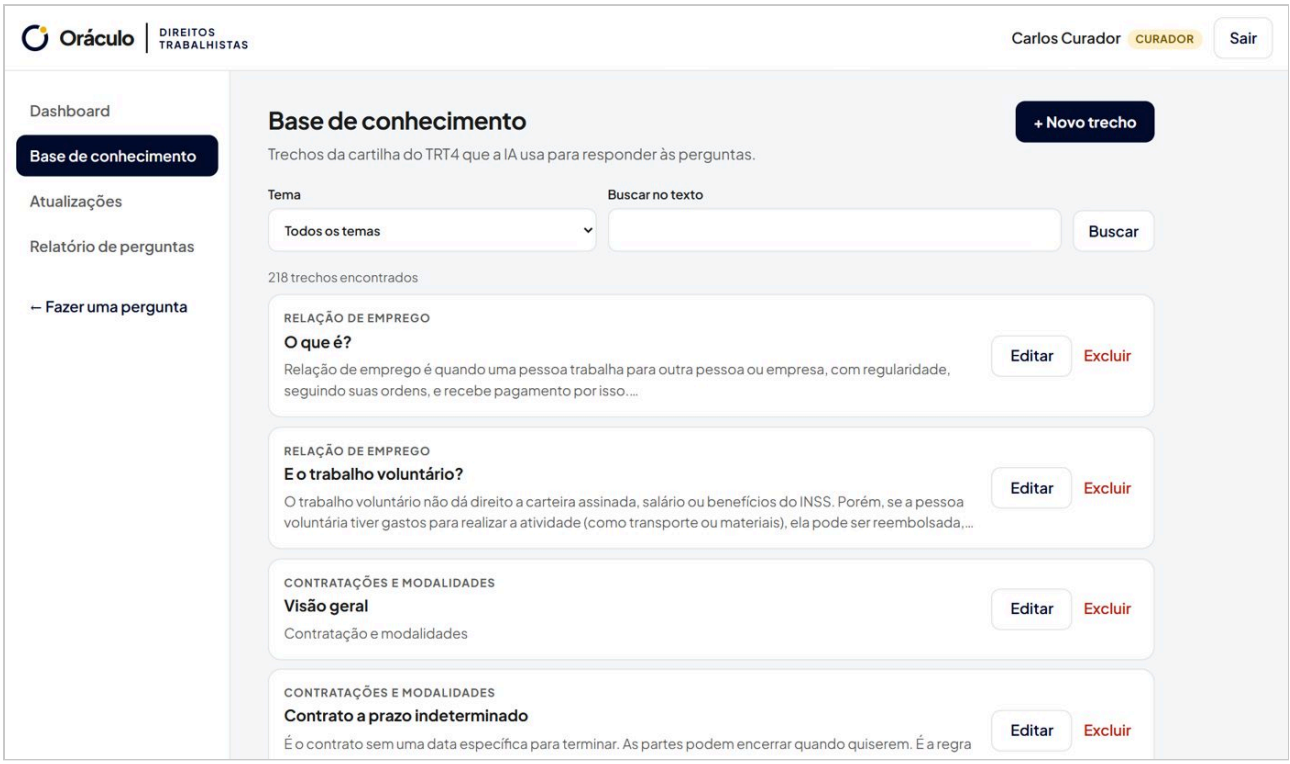
Cadastro concluído (conta gravada no banco)

Controle de perfis: usuário comum sem acesso à base



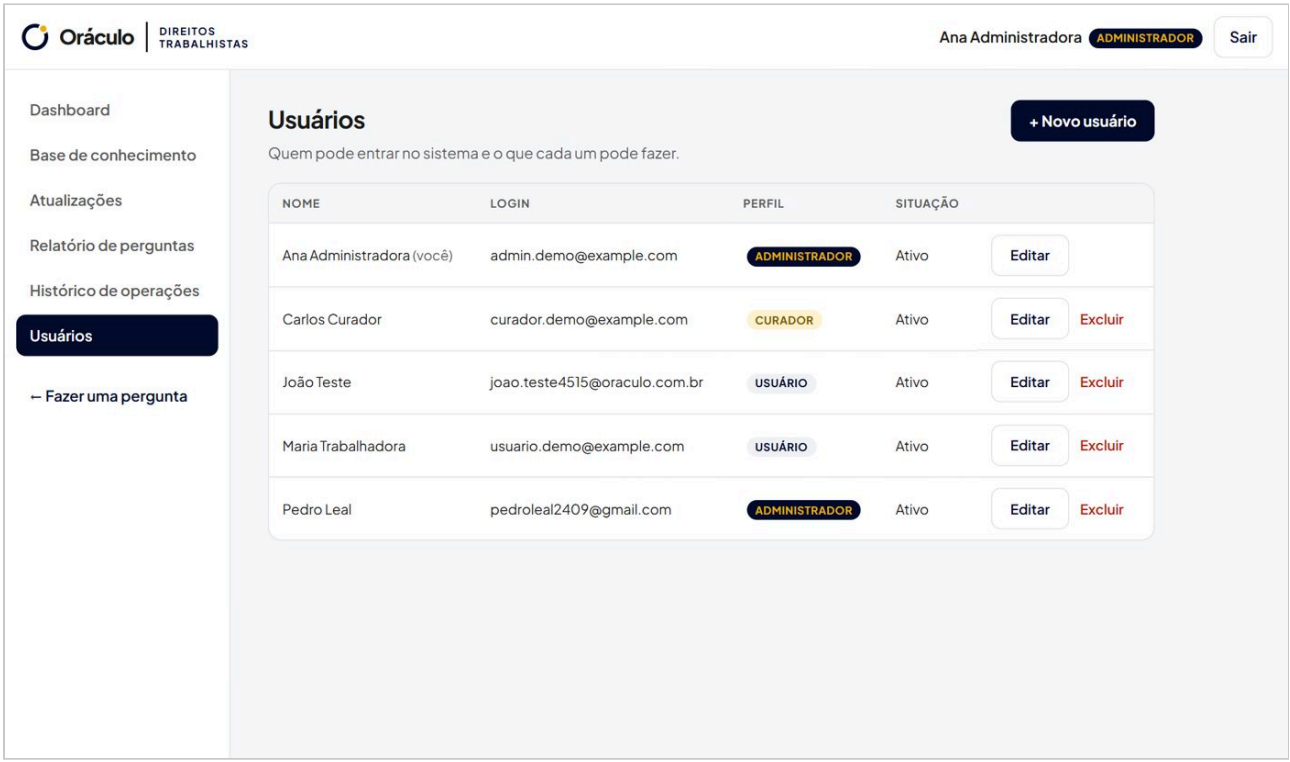
Controle de perfis: usuário comum sem acesso à base

CRUD: consulta da base de conhecimento (dados do Supabase)



CRUD: consulta da base de conhecimento (dados do Supabase)

Gestão de usuários e perfis (administrador)



Gestão de usuários e perfis (administrador)

Como executar localmente

1. Configure as contas e o arquivo `backend/.env` (guia).
2. Rode `iniciar.bat` (Windows) ou `./iniciar.sh` (Linux/macOS).
3. Crie o primeiro administrador: `backend\.venv\Scripts\python backend\scripts\criar_administrador.py`.

Mudanças em relação ao planejado

- O login foi implementado no próprio banco (tabela de usuários e sessões), em vez do Supabase Auth, para que o login, os perfis e as sessões fiquem integrados às tabelas do projeto.
- A consulta pública continua sem login (RF01); a conta é usada pela equipe e por quem quer acompanhar temas e receber as novidades por e-mail (ver Sprint 05).

Sprint 04 — Módulo 1: gestão da base de conhecimento

Situação: Concluída e testada com o banco real

O que a Sprint pediu

Primeiro módulo totalmente funcional, com persistência de dados, validações, mensagens de erro claras, navegação entre telas e commits organizados.

Módulo entregue: gestão da base de conhecimento

A equipe (administradores e curadores) mantém os trechos da cartilha que a IA usa para responder.

Fluxo completo: entrar → ver a base (filtrar por tema e pesquisar no texto) → cadastrar um trecho → editar → excluir com confirmação. Tudo fica gravado no Supabase e continua disponível depois de fechar e abrir o sistema.

O que foi entregue

Item pedido	Entrega	Exemplos
Módulo funcional	CRUD da base de conhecimento ligado ao banco, com os sinônimos da busca e os sites e vídeos do "Saber mais"	Telas **Base de conhecimento**, **Sinônimos da busca** e **Sites e vídeos**
Persistência	Dados no Supabase (PostgreSQL); o roteiro de evidências reinicia o sistema e confere que o trecho continua salvo	`docs/ferramentas/capturar_evidencias.py`
Validações	No navegador (antes de enviar) e no servidor (regras iguais)	Subtítulo de 2 a 200 caracteres; texto de 10 a 5000; tema existente; e-mail válido; senha com no mínimo 8 caracteres
Mensagens de erro	Erros técnicos viram frases em português, junto do campo	"Preencha o campo subtítulo.", "Escolha um tema válido.", "Sem conexão com o servidor..."
Navegação	Rotas com menu por perfil; sem login vai para Entrar; perfil sem permissão vê um aviso; página 404	`frontend/src/App.jsx`
Commits organizados	Um commit por ajuste, com data, objetivo, o que foi feito e como agrega	Histórico no GitHub

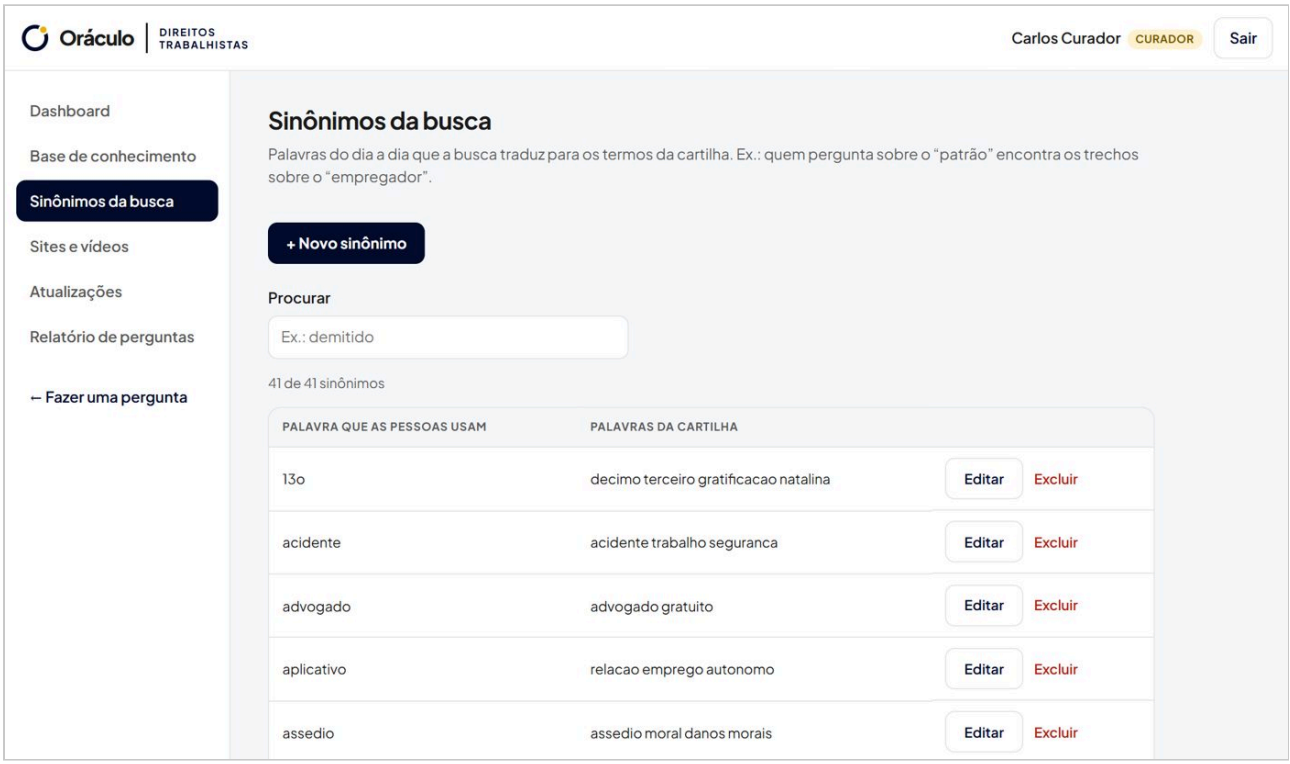
Ajustes desta Sprint

Commit	O que mudou
`5b2f043`	Tradução das mensagens de validação para o usuário
`e5c21bf`	Navegação do painel, validações nos formulários e confirmação antes de excluir
`4165443`	Repetição automática quando a conexão com o banco cai; mensagem clara para erros inesperados

Evidências

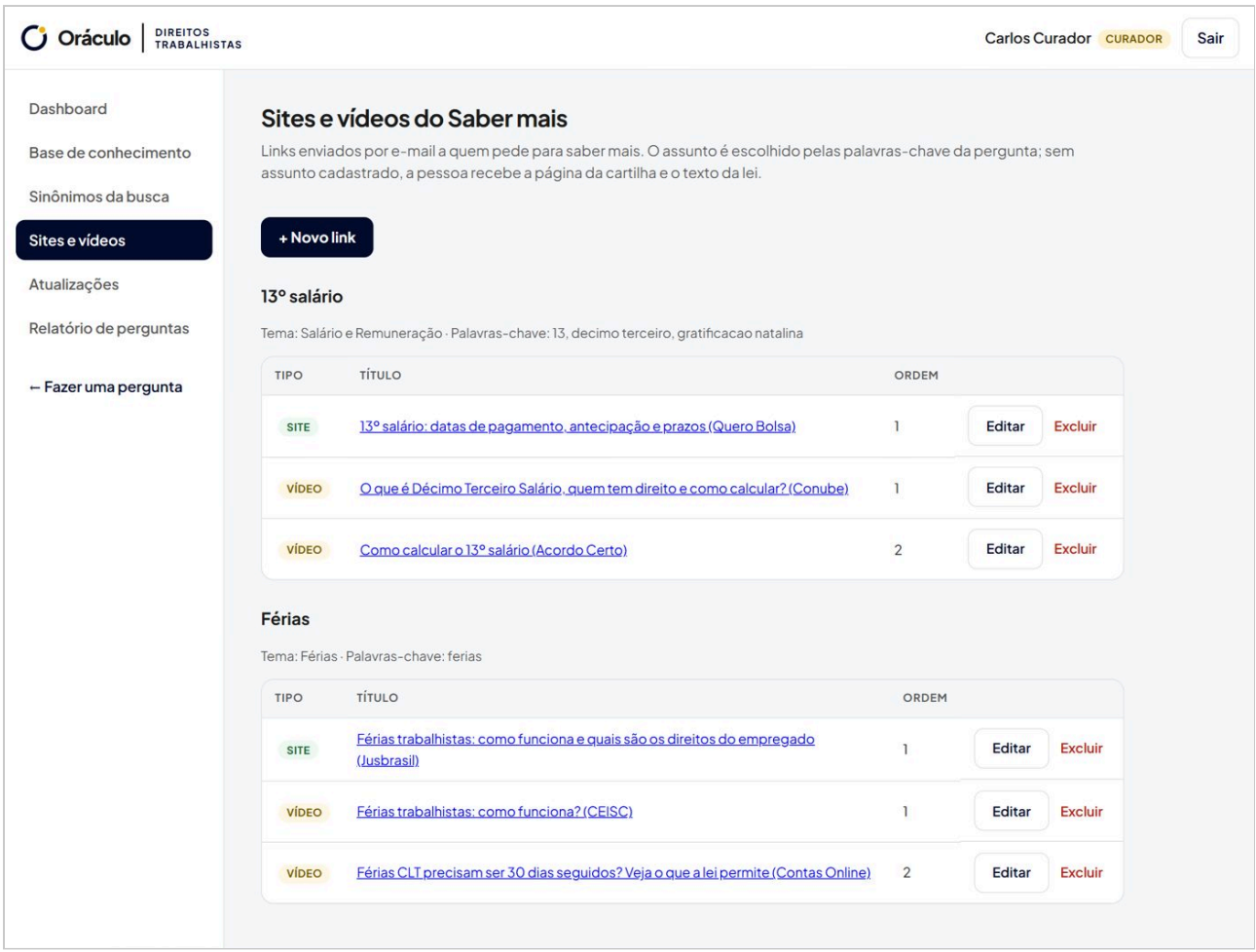
Prints gerados em 02/11/2026 pelo roteiro automatizado `capturar_evidencias.py`, com o sistema rodando localmente e o banco real no Supabase. Todos os prints estão em `docs/evidencias/`.

Sinônimos da busca: palavras do dia a dia traduzidas para os termos da cartilha



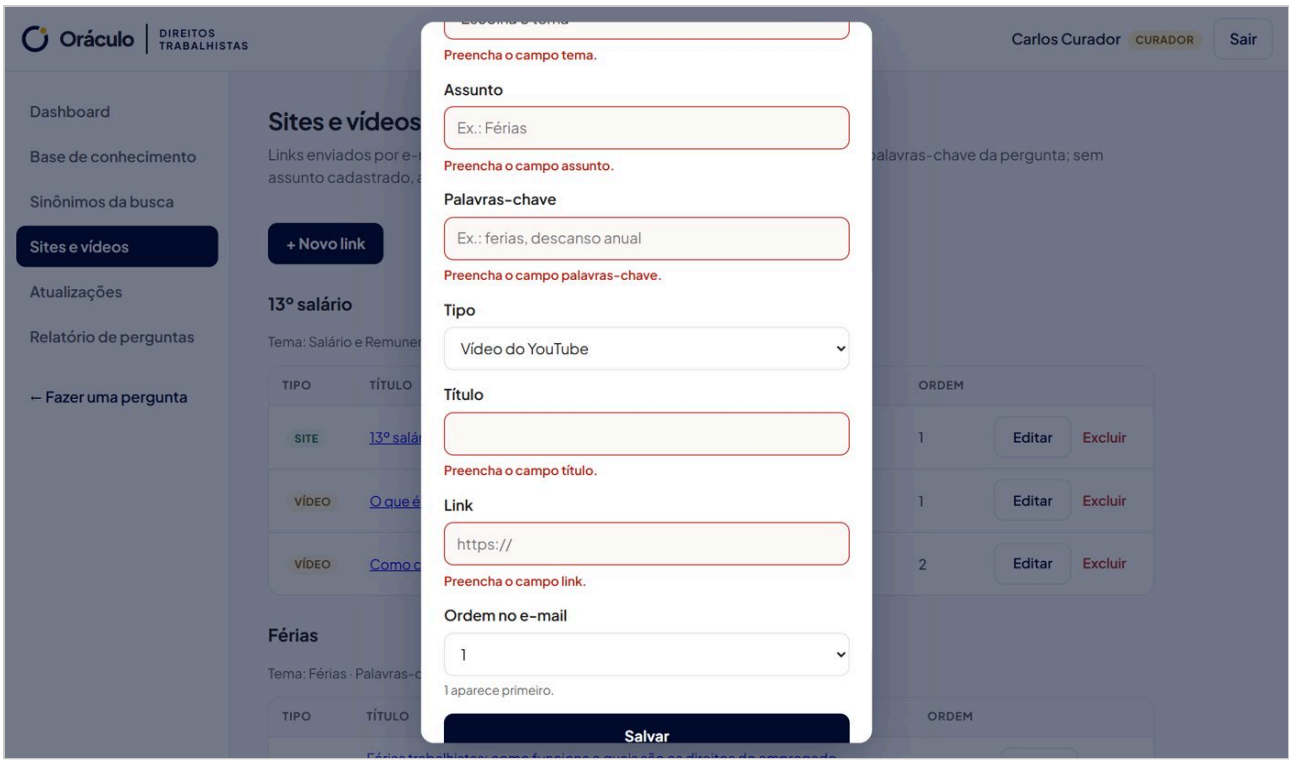
Sinônimos da busca

Sites e vídeos do "Saber mais", agrupados por assunto



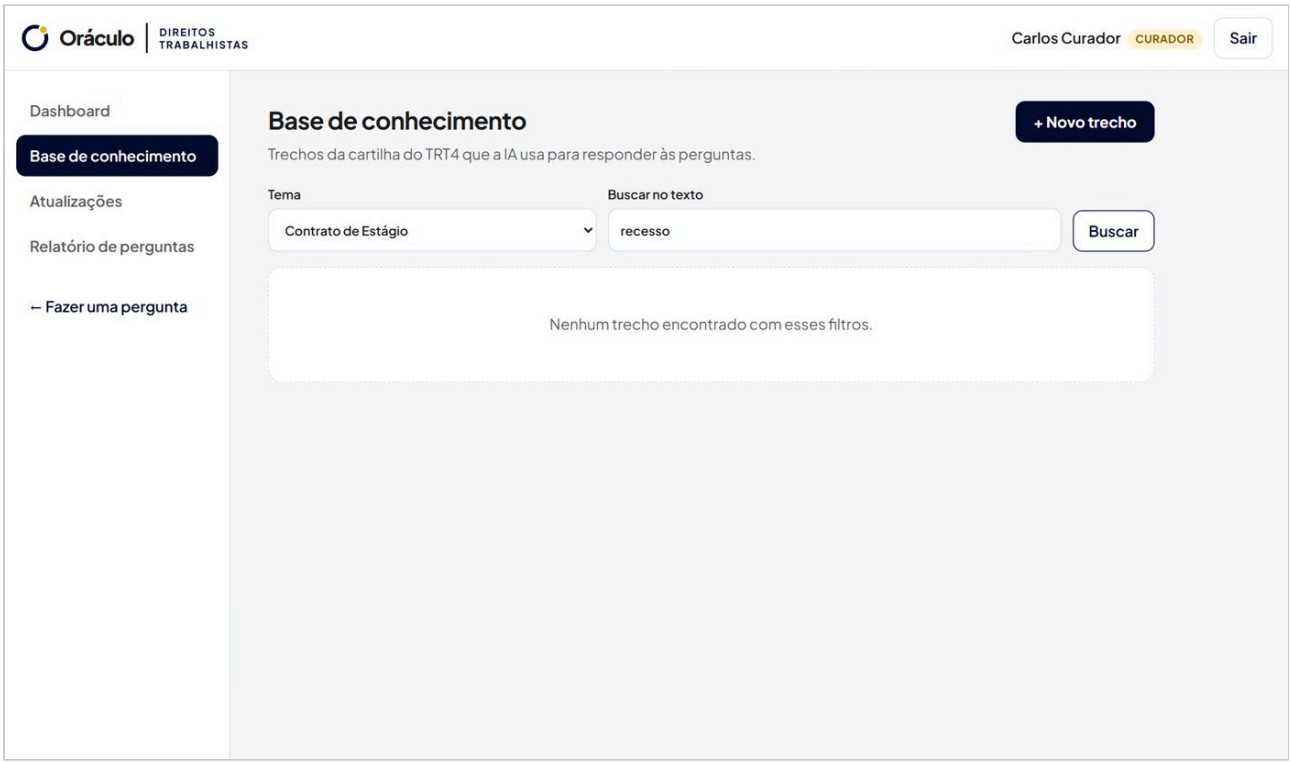
Sites e vídeos do Saber mais

Validação do cadastro de um link



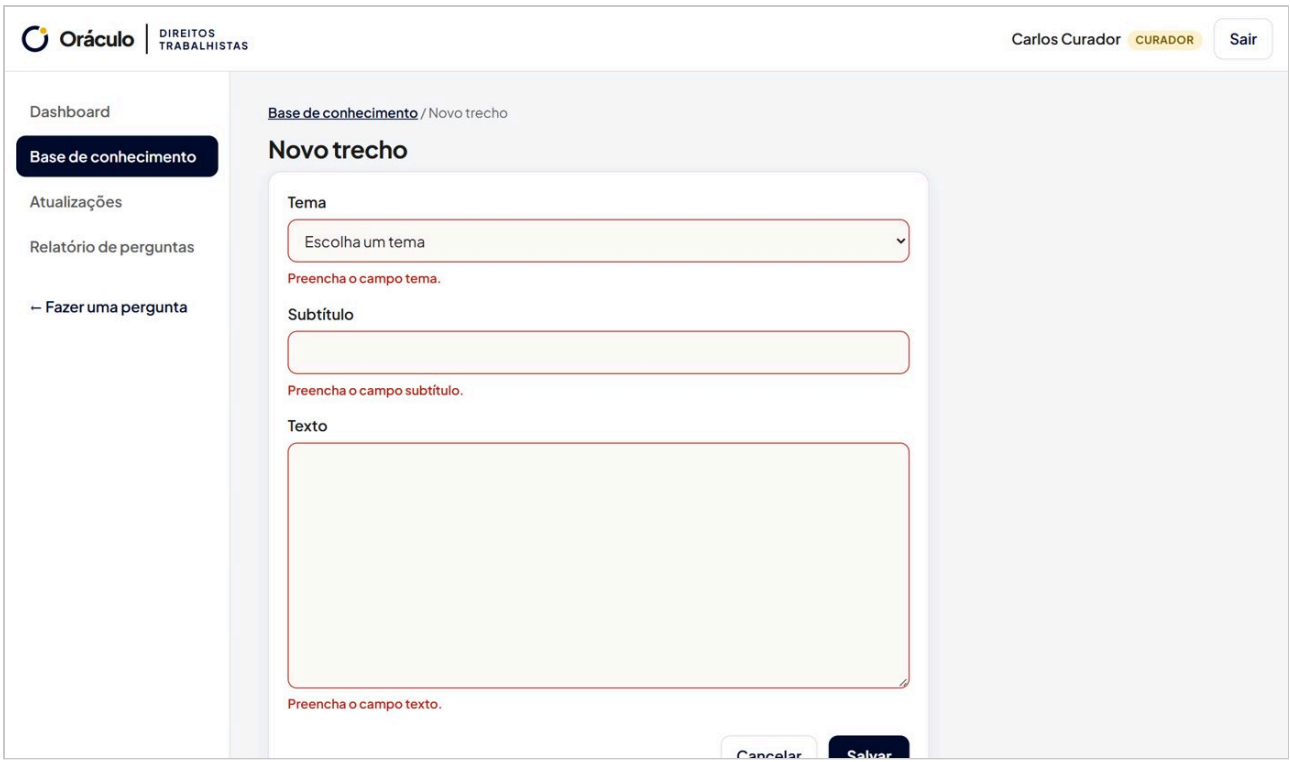
Validação do cadastro de um link

Consulta com filtro por tema e pesquisa no texto



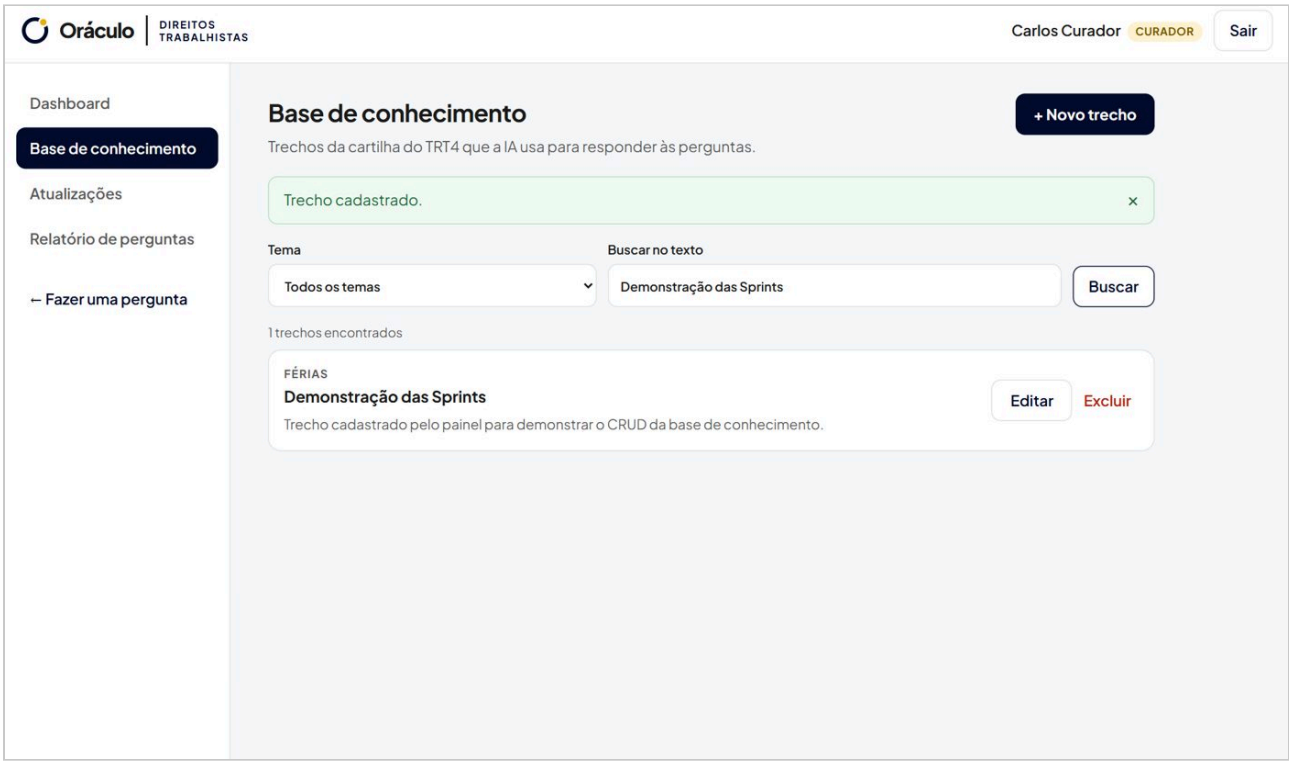
Consulta com filtro por tema e pesquisa no texto

Validação: campos obrigatórios do trecho



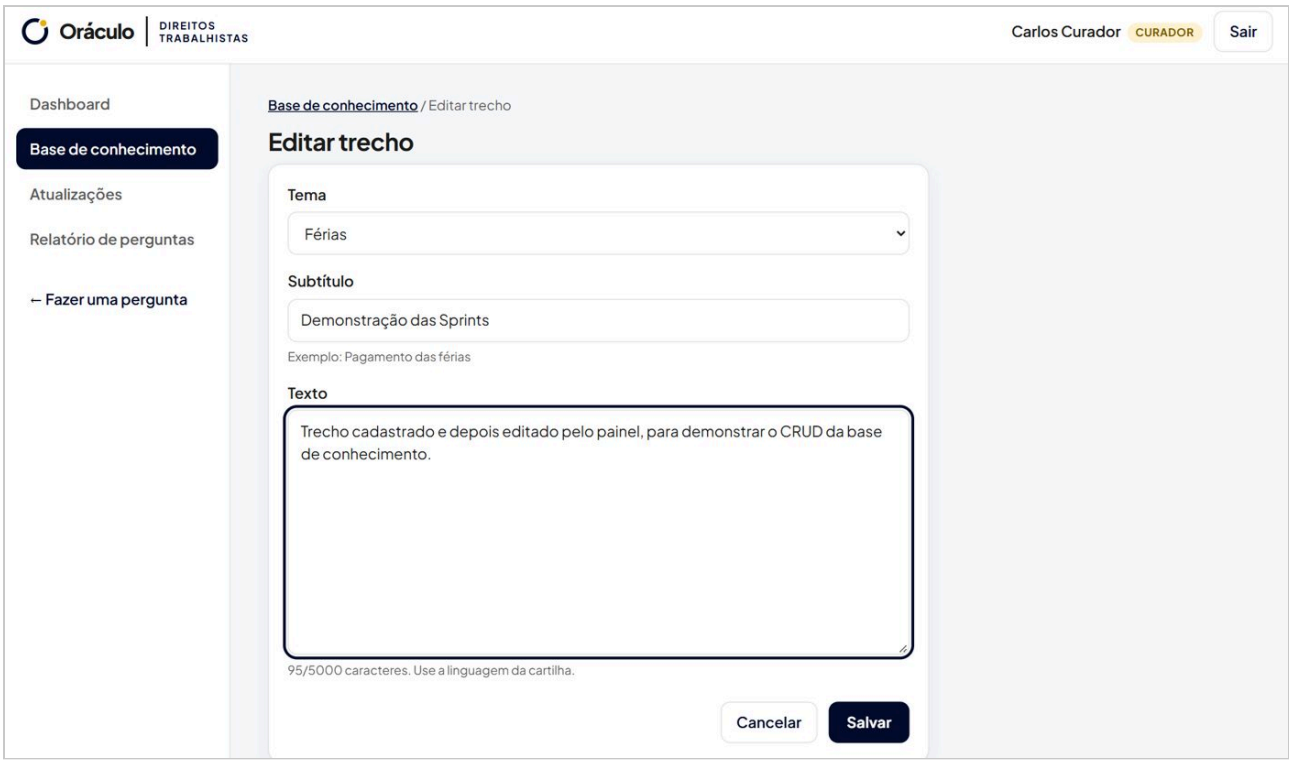
Validação: campos obrigatórios do trecho

Cadastrar: trecho gravado e encontrado na pesquisa



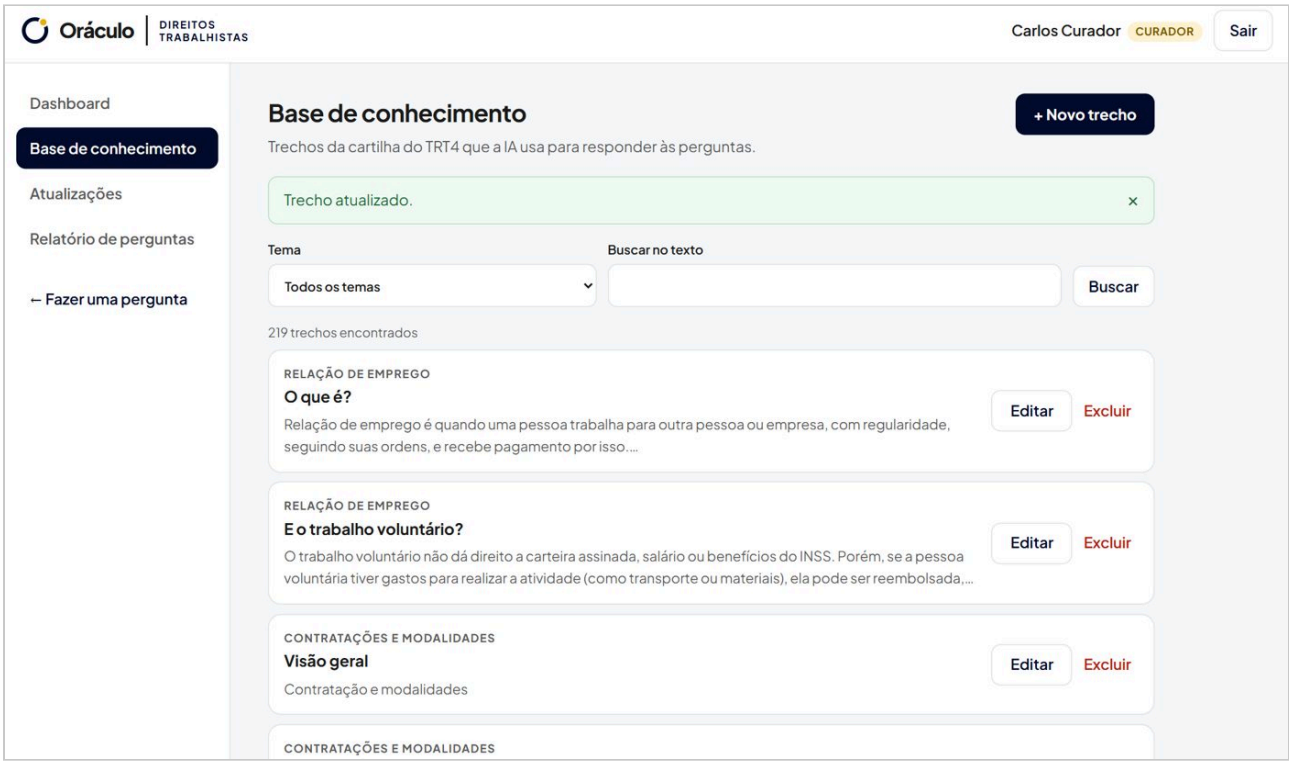
Cadastrar: trecho gravado e encontrado na pesquisa

Atualizar: edição do trecho



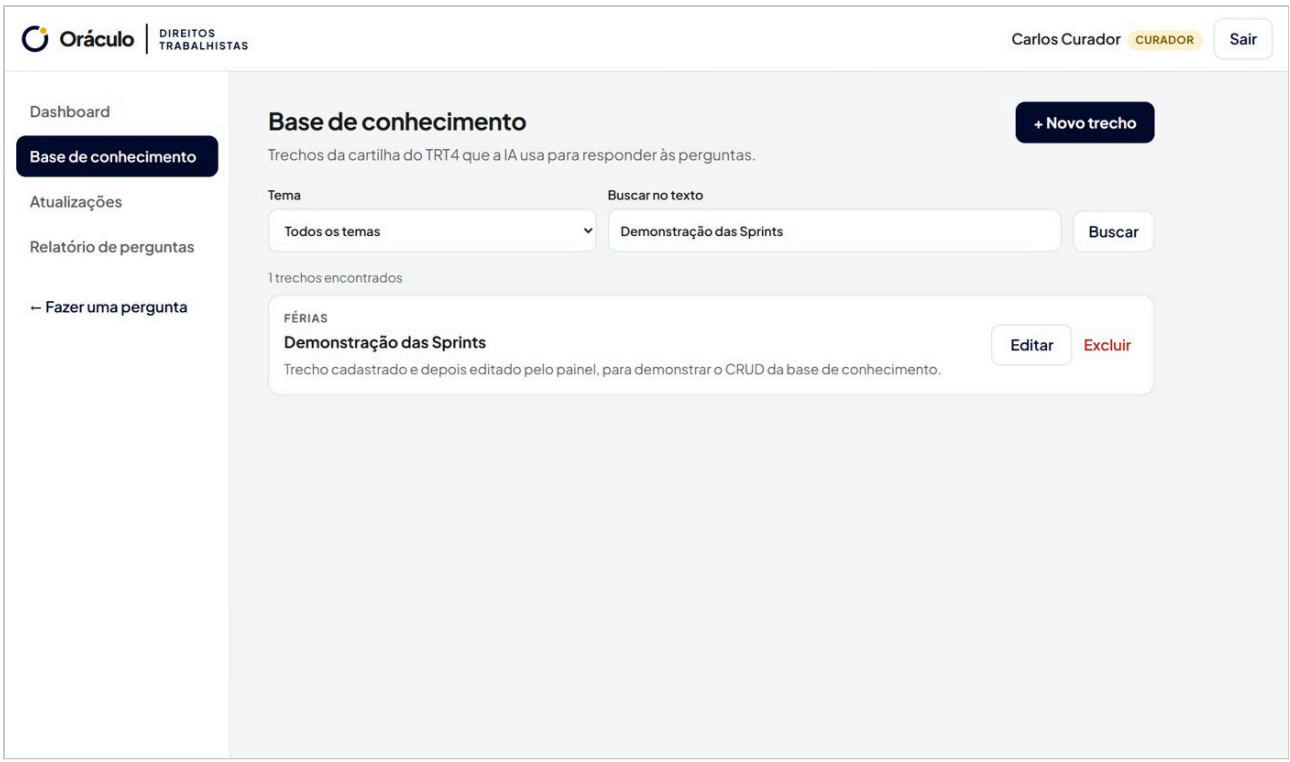
Atualizar: edição do trecho

Mensagem de sucesso após salvar



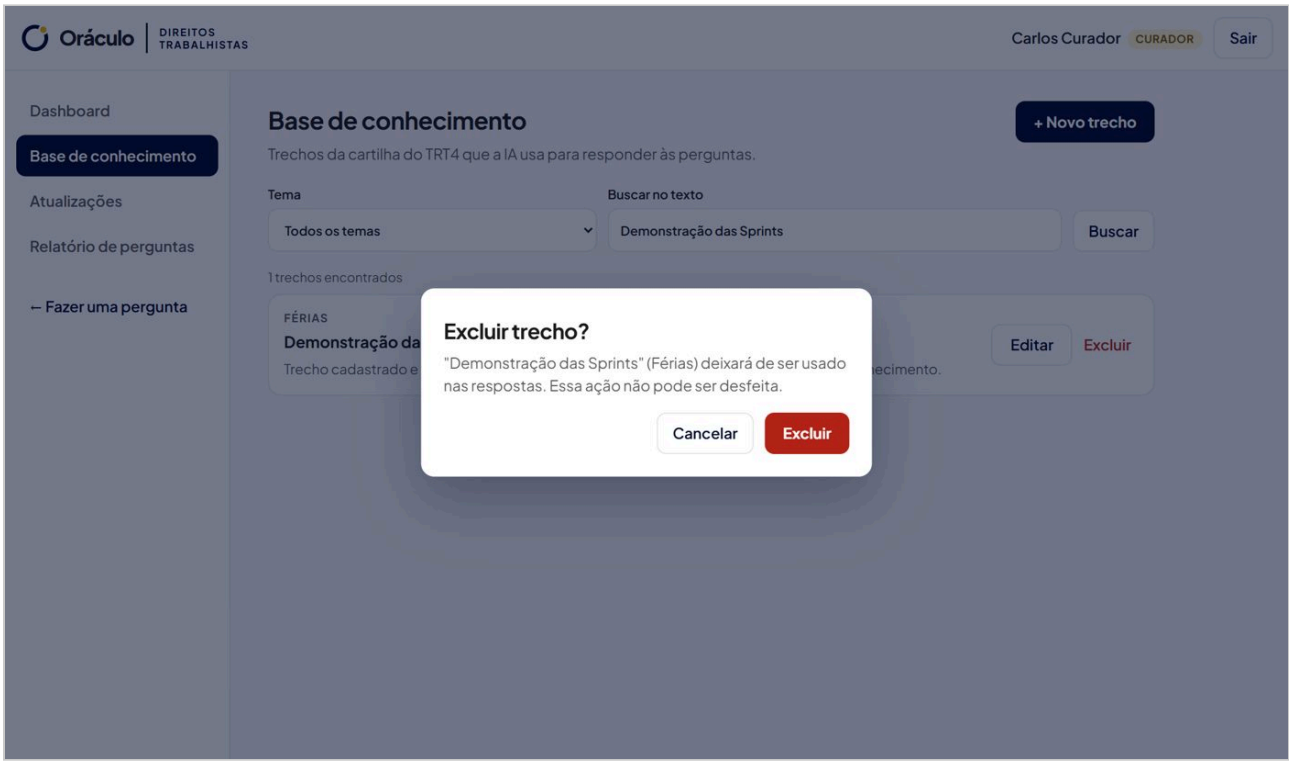
Mensagem de sucesso após salvar

Persistência: o trecho continua salvo depois de fechar e abrir o sistema



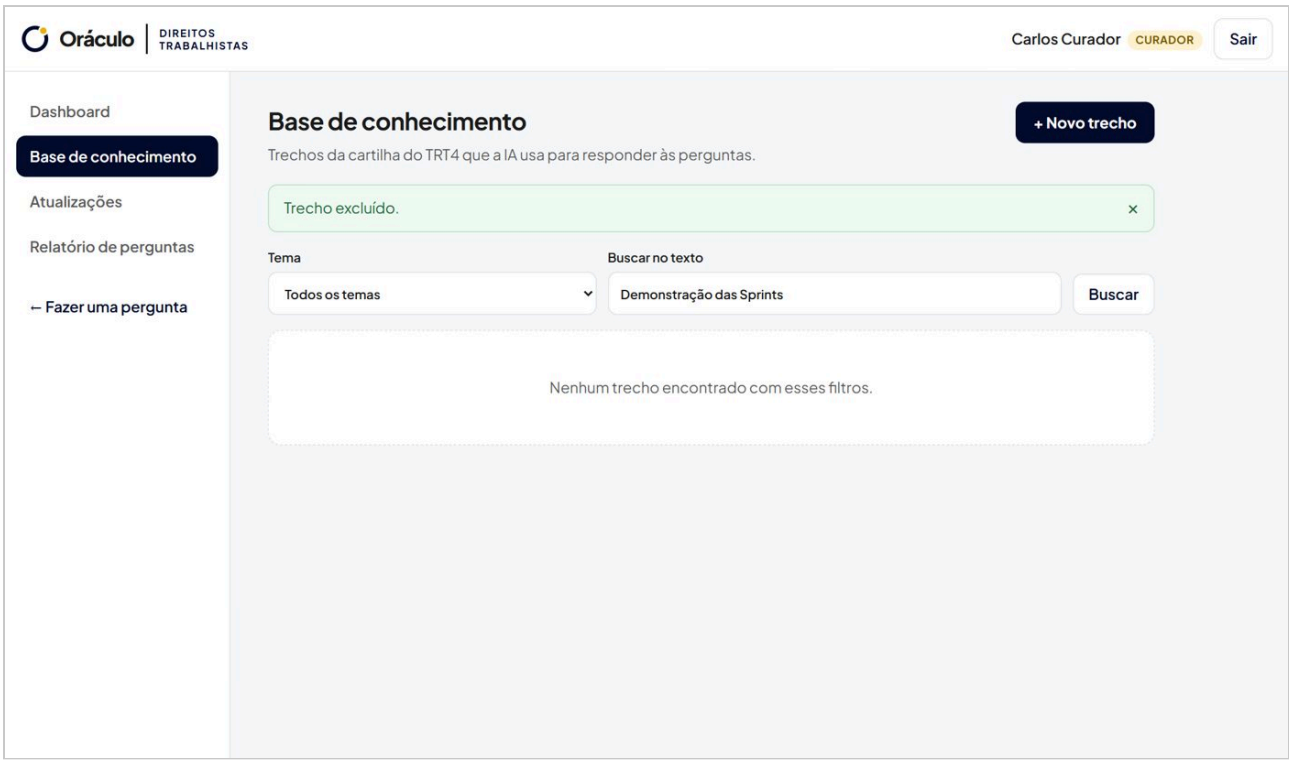
Persistência: o trecho continua salvo depois de fechar e abrir o sistema

Excluir: confirmação antes de apagar



Excluir: confirmação antes de apagar

Excluir: trecho removido



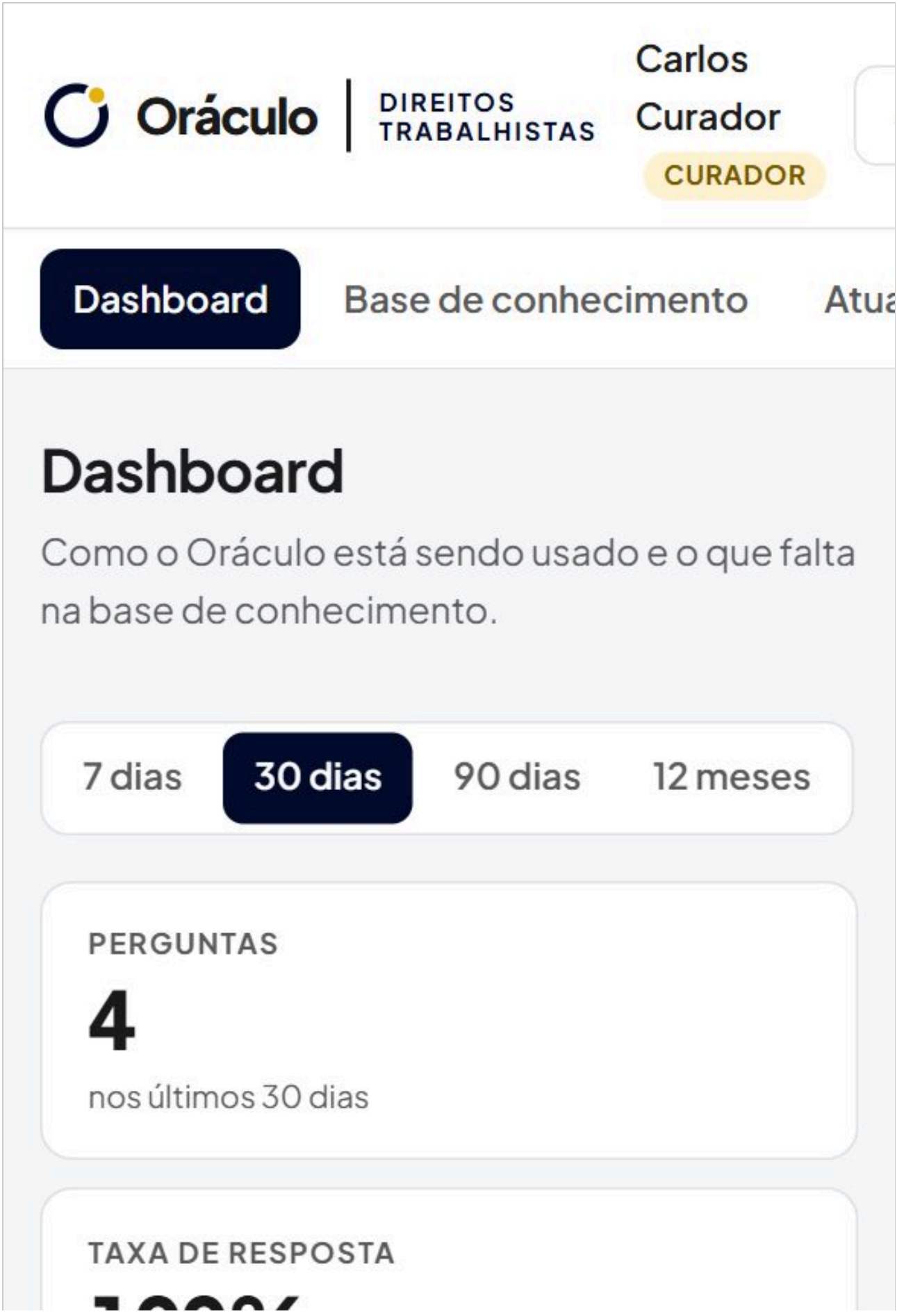
Excluir: trecho removido

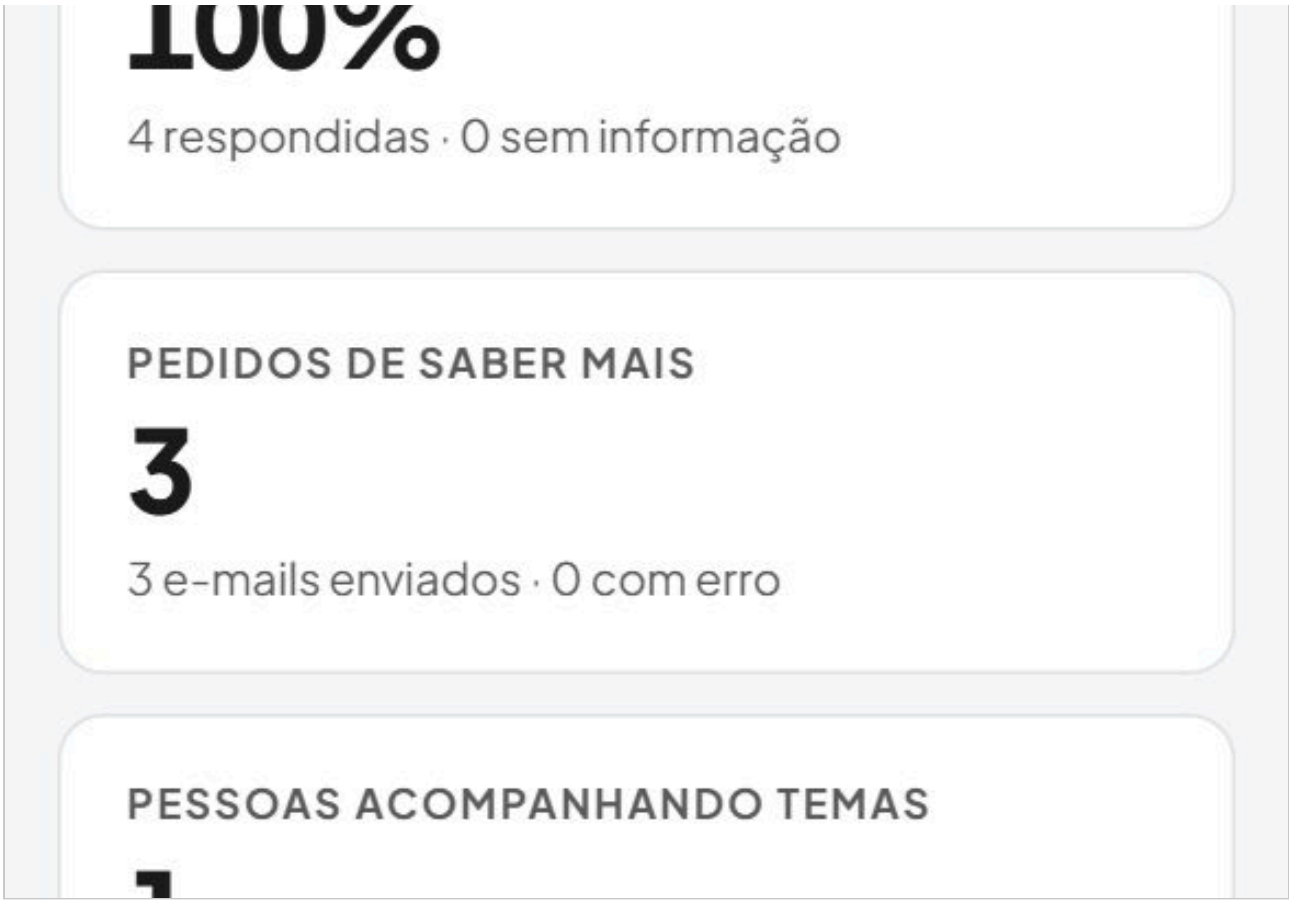
Mensagens de erro junto de cada campo



Mensagens de erro junto de cada campo

Navegação no celular


Oráculo
DIREITOS TRABALHISTAS
Carlos Curador
CURADOR
Dashboard
Base de conhecimento
Atua
Dashboard
Como o Oráculo está sendo usado e o que falta na base de conhecimento.
7 dias
30 dias
90 dias
12 meses
PERGUNTAS
4
nos últimos 30 dias
TAXA DE RESPOSTA



Navegação no celular

Sprint 05 — Módulo 2: acompanhamento de temas

Situação: Concluída e testada com o banco real e com o envio real de e-mail

O que a Sprint pediu

Segundo módulo completo, integrado ao banco de dados, com regras de negócio atualizadas, testes das funcionalidades e correção dos bugs encontrados.

Módulo entregue: acompanhamento de temas por e-mail

Fluxo completo: a pessoa cria a conta com o e-mail → escolhe temas para acompanhar → a equipe publica uma atualização do tema → o sistema envia um e-mail a cada inscrito e registra a situação de cada envio → a pessoa também vê a novidade em **Meus temas**.

O que foi entregue

Item pedido	Entrega	Onde ver
Segundo módulo	Conta com e-mail, temas acompanhados e atualizações	Telas **Crie sua conta**, **Meus temas** e **Atualizações**
Integração com o banco	Tabelas `usuarios`, `inscricoes_tema`, `atualizacoes` e `envios_atualizacao`	`supabase/003_modelo_relacional.sql` e `006_contato_por_email.sql`
Regras de negócio	Ver tabela abaixo	`rotas/atualizacoes.py`
Testes	Testes automatizados no backend (`pytest`), 88 no total na versão atual	`backend/tests/`
Correção de bugs	Ver tabela de bugs abaixo	—

Regras de negócio implementadas ou atualizadas

Regra	Justificativa
O cadastro público usa **nome, e-mail e senha**, com autorização obrigatória (LGPD)	A conta existe para a pessoa acompanhar temas e receber as novidades por e-mail
Todos entram com **e-mail e senha**	Um único jeito de entrar, simples para todos os perfis
Só recebem a atualização as **inscrições ativas** do tema no momento da publicação	Evita e-mails a quem deixou de acompanhar
O **curador só exclui as próprias** publicações; o administrador exclui qualquer uma	Responsabilidade sobre o que foi publicado

Regra	Justificativa
A publicação é salva **mesmo sem o e-mail configurado**; os envios ficam com erro e podem ser reenviados	Nenhuma informação se perde por falha externa
Uma pessoa não acompanha o mesmo tema duas vezes	Evita e-mails duplicados

Bugs encontrados e corrigidos

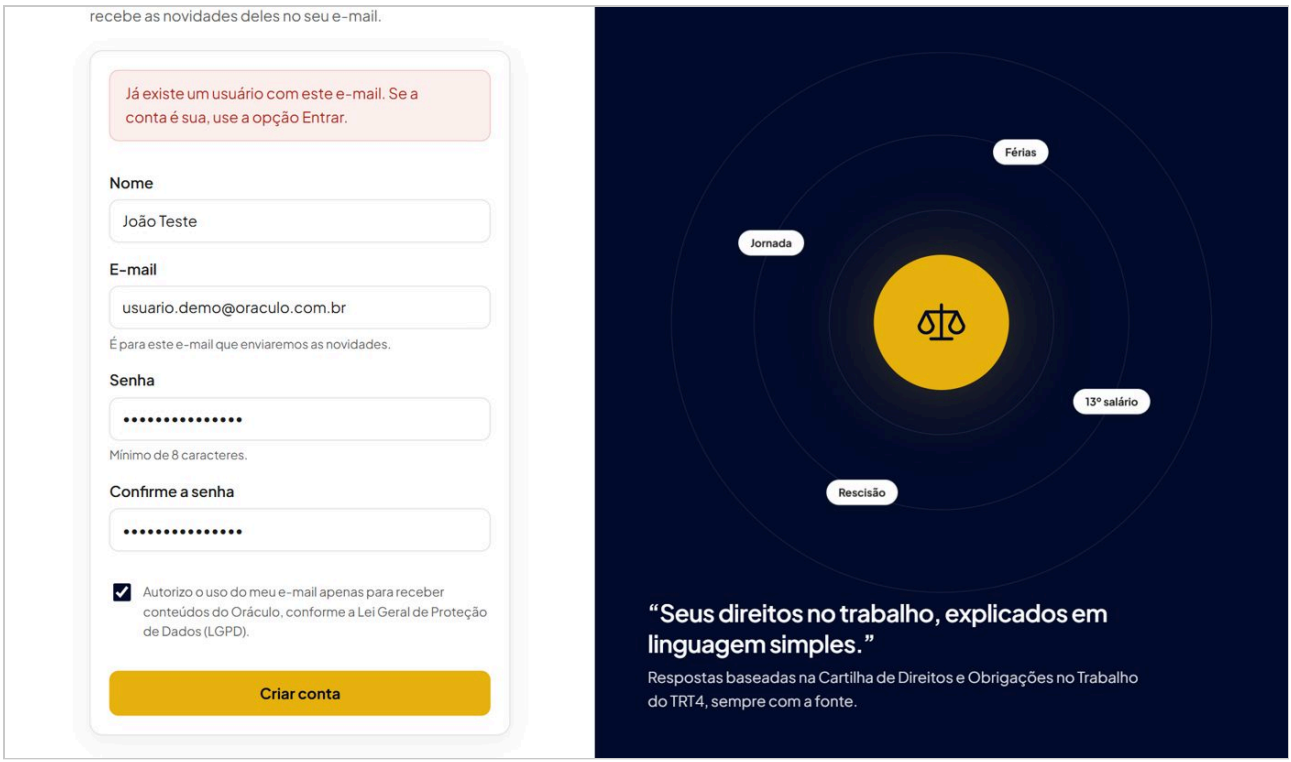
Bug	Causa	Correção
A busca não encontrava "estágio" nem com a palavra exata	A função SQL processava as palavras da pergunta duas vezes	Função corrigida (`002`)
"Demitido", "patrão" e "FGTS" não achavam os temas certos	A cartilha usa outras palavras ("dispensa", "empregador")	Tabela de sinônimos editável
Erro 500 intermitente ao abrir a base	O Supabase encerra conexões paradas	Repetição automática da operação (`4165443`)
Cadastros de teste recusados	O validador de e-mail não aceita o domínio reservado `.test`	Testes com domínio válido
Filtro de período do dashboard sempre recusado	O valor chega como texto e não era convertido	Validação própria com mensagem clara

O registro completo de bugs, incluindo os encontrados depois, está na Sprint 09.

Evidências

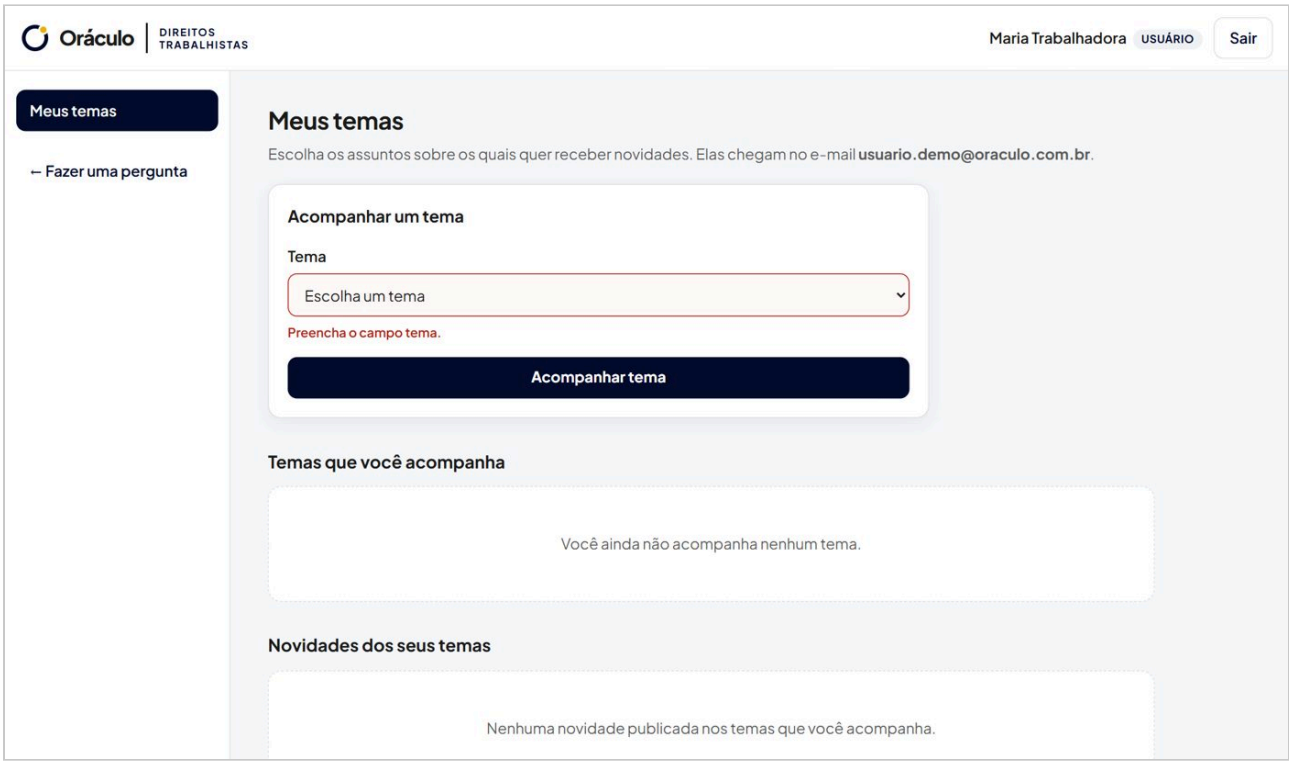
Prints gerados em 02/11/2026 pelo roteiro automatizado `capturar_evidencias.py`, com o sistema rodando localmente e o banco real no Supabase. Todos os prints estão em `docs/evidencias/`.

Regra: não pode haver duas contas com o mesmo e-mail



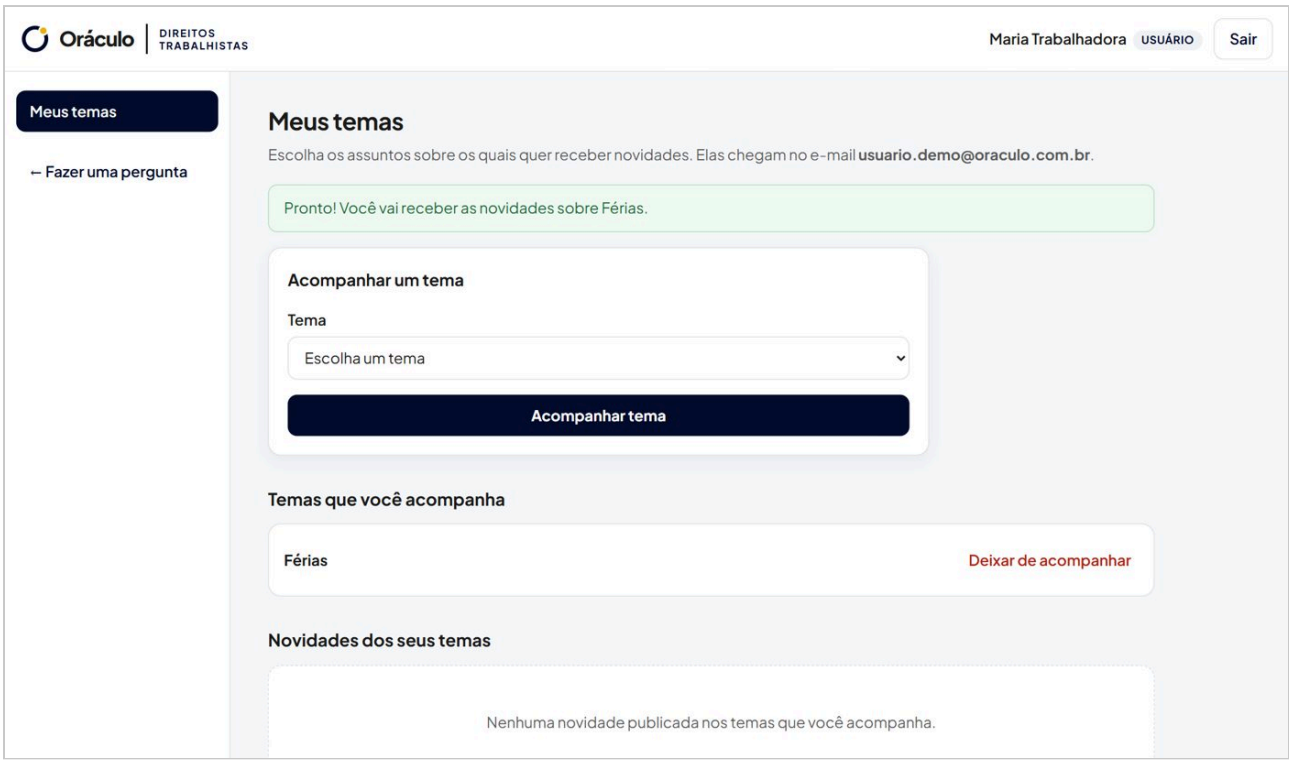
Regra: não pode haver duas contas com o mesmo e-mail

Meus temas: validação



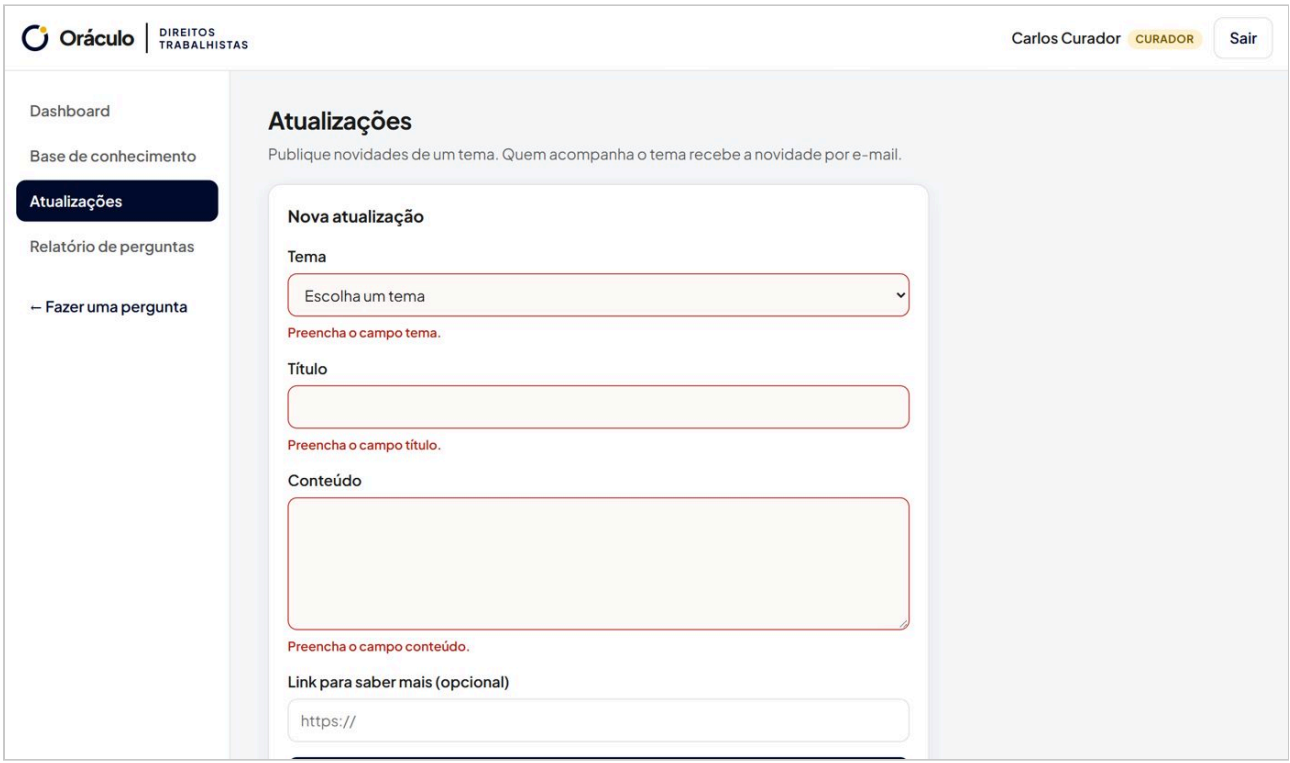
Meus temas: validação

Meus temas: tema acompanhado (gravado no banco)



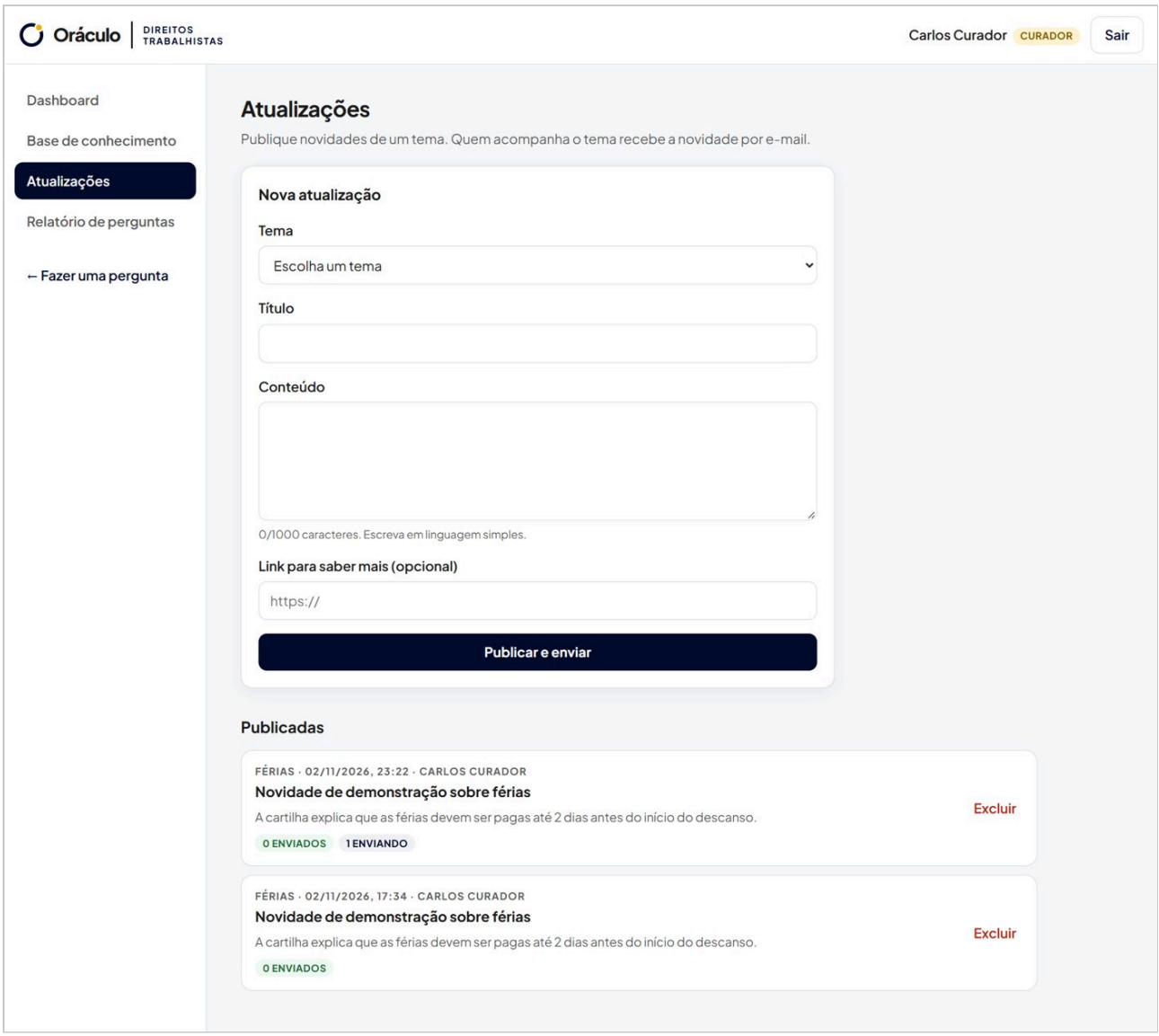
Meus temas: tema acompanhado (gravado no banco)

Atualizações: validação do formulário



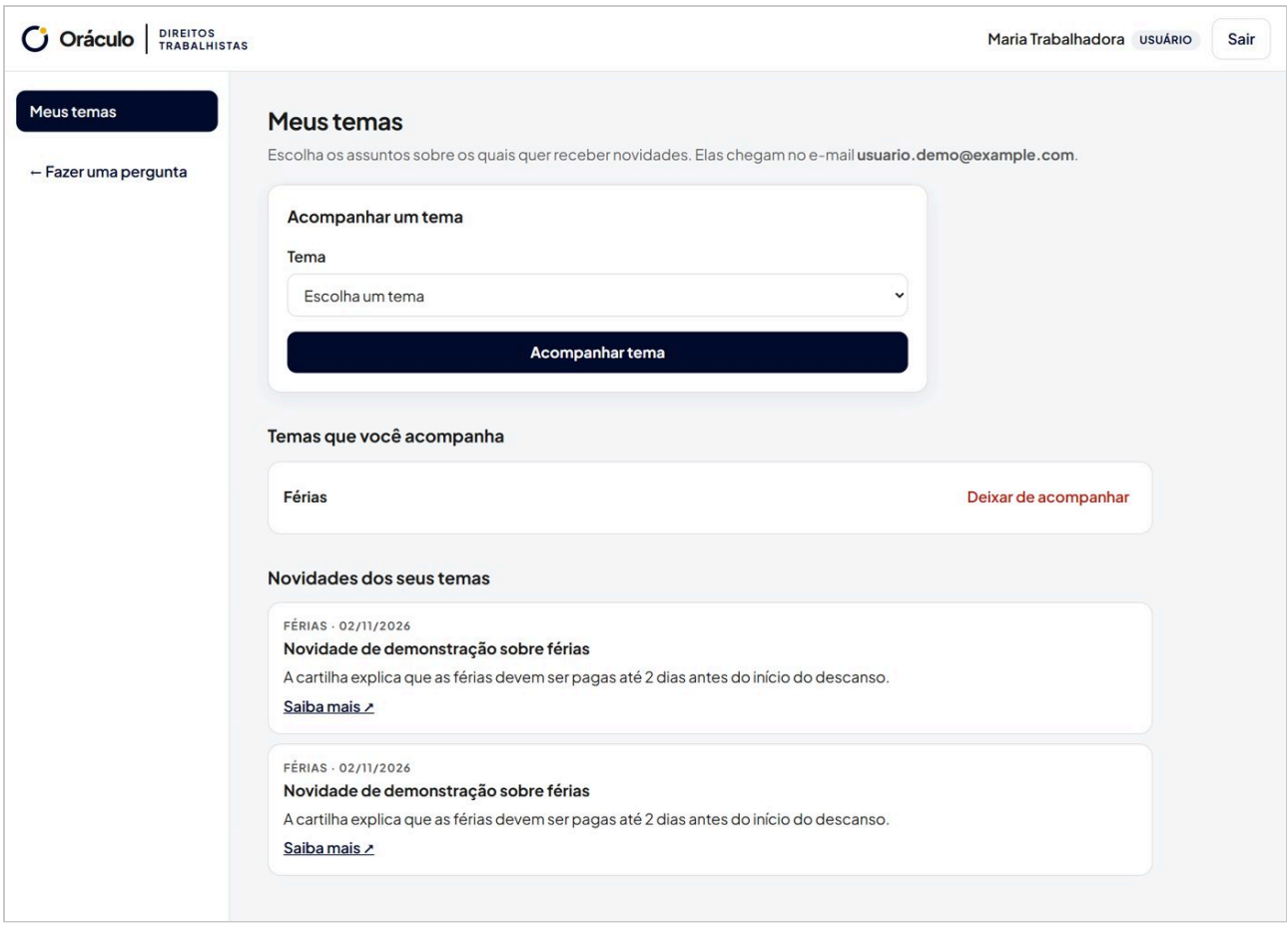
Atualizações: validação do formulário

Atualização publicada; sem o e-mail configurado o envio fica com erro e pode ser reenviado



Atualização publicada; sem o e-mail configurado o envio fica com erro e pode ser reenviado

A pessoa vê a novidade do tema que acompanha



A pessoa vê a novidade do tema que acompanha

Ajustes desta Sprint

Commit	O que mudou
`43414b5`	Cadastro e login da conta; "Saiba mais" e temas ligados à conta
`7f76733`	Telas de conta e "Saiba mais" com criar conta / entrar
`7f43866`	Módulo 2 no backend: publicar, enviar, reenviar e excluir atualizações
`e24c793`	Telas do Módulo 2: Atualizações e novidades em Meus temas

Sprint 06 — Módulo 3, integração e melhorias de interface

Situação: Concluída: módulo 3 e integração prontos e testados com a IA real; Pré-Banca dispensada pela orientadora

O que a Sprint pediu

Correções apontadas na Pré-Banca, terceiro módulo implementado, integração entre os módulos, melhorias na interface e ajustes de navegação.

Módulo entregue: consulta inteligente (componente de IA)

Fluxo: a pessoa faz a pergunta → o sistema busca os trechos mais relevantes da cartilha no Supabase → a IA (Google Gemini) escreve a resposta usando só esses trechos → a resposta aparece com as fontes → em "Quer saber mais? Sim", a pessoa informa nome e e-mail e recebe a resposta, 2 sites e 2 vídeos.

Integração entre os módulos

De	Para	Como
Módulo 1 (base de conhecimento)	Módulo 3 (consulta)	Todo trecho cadastrado ou editado pela equipe passa a ser usado pela IA nas respostas
Módulo 3 (consulta)	Módulo 1 (base)	Perguntas sem resposta aparecem no Dashboard e no relatório com o botão **Incluir na base**
Módulo 2 (conta)	Módulo 3 (consulta)	Quem já entrou na conta tem o nome e o e-mail preenchidos no "Saber mais"; quem não tem conta só informa nome e e-mail
Todos	Histórico e Dashboard	Operações registradas no histórico e somadas nos indicadores (Sprint 07)

O que foi entregue

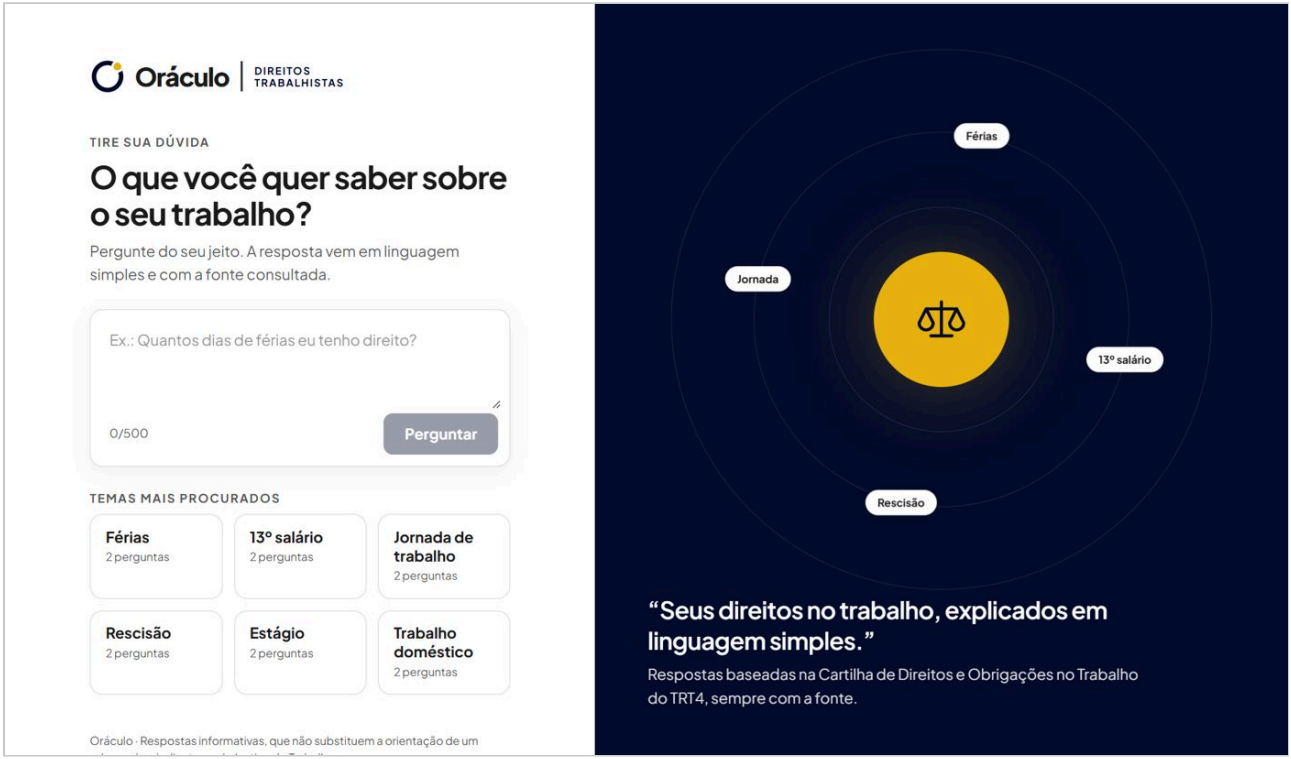
Item pedido	Situação
Correções da Pré-Banca	**Não se aplica:** a equipe apresentou o projeto à orientadora na semana de 02 a 06/11/2026, e ela dispensou a Pré-Banca. A orientadora avaliou o projeto como ótimo e pediu apenas uma conversa com o outro orientador para confirmar o uso de paralelismo (Sprint 08)
Terceiro módulo	Implementado e testado com a IA real (Google Gemini) e o envio real de e-mail
Integração entre módulos	Concluída (tabela acima)
Melhorias na interface	Abaixo da resposta, no painel azul (com o símbolo da órbita), o convite "**Quer saber mais sobre o assunto?**" com o botão **Sim**; ao clicar, abre a janela **Saber mais**, que explica o que será enviado e pede só nome, e-mail e a autorização (LGPD); menu por perfil

Item pedido	Situação
Ajustes de navegação	Cada perfil começa pela sua área (equipe → Dashboard, usuário → Meus temas); depois de sair, a próxima pessoa não é mais levada à página da anterior (bug B06); "Meus temas" só aparece para quem tem conta de usuário

Evidências

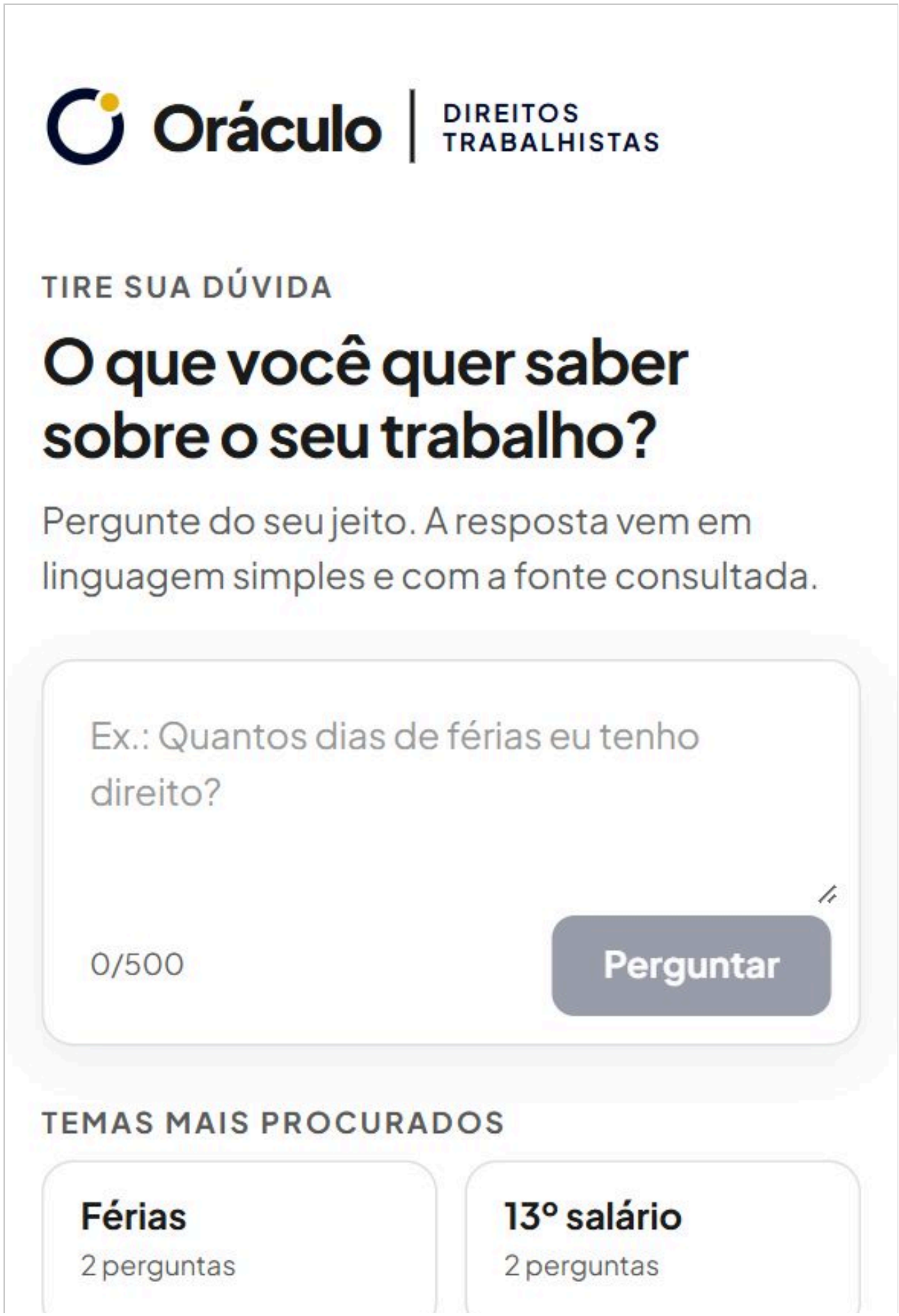
Prints gerados em 02/11/2026 pelo roteiro automatizado `capturar_evidencias.py`, com o sistema rodando localmente e o banco real no Supabase. Todos os prints estão em `docs/evidencias/`.

Consulta: tela inicial com o painel de órbita



Consulta: tela inicial com o painel de órbita

Consulta no celular

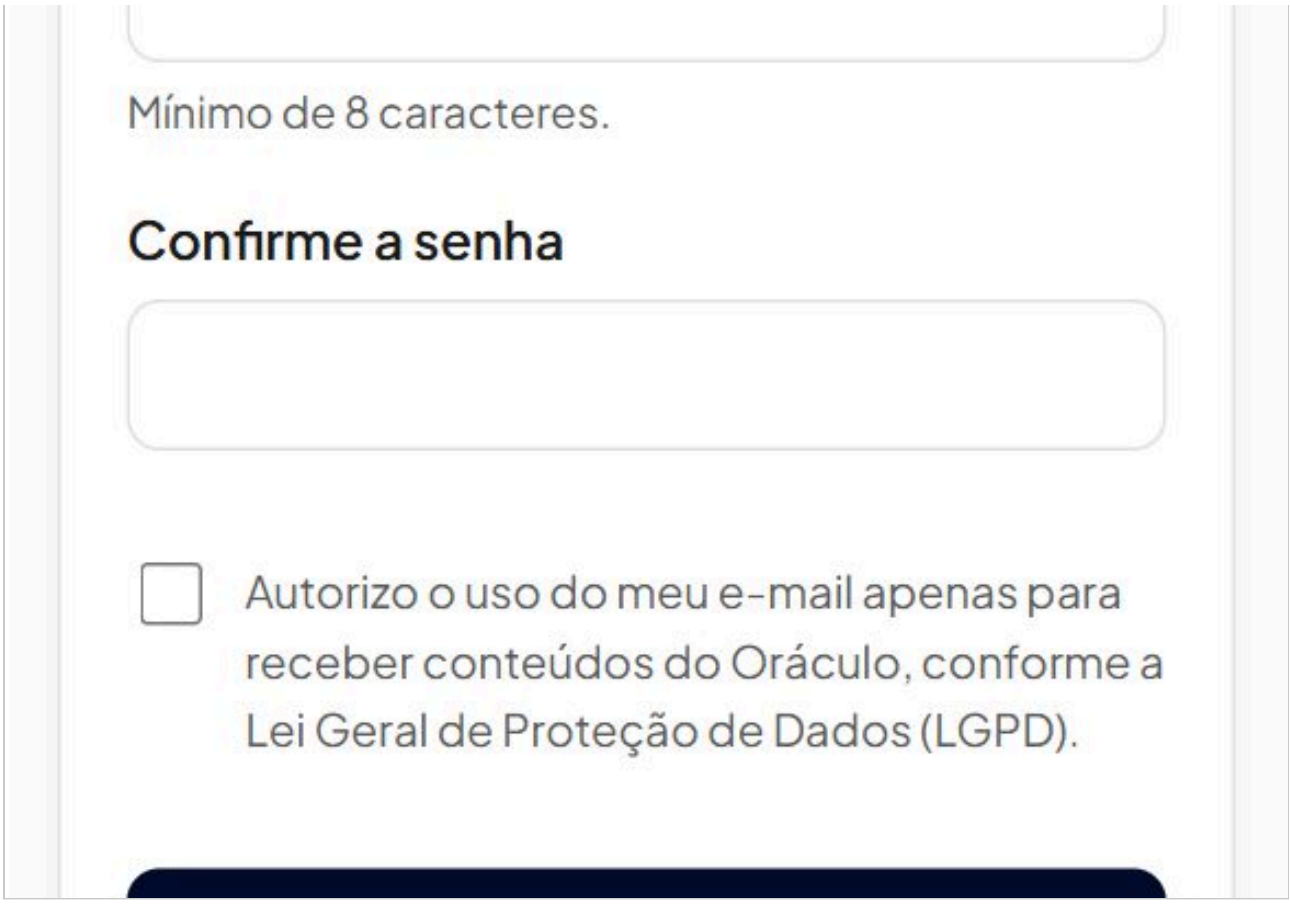

Oráculo
DIREITOS TRABALHISTAS
TIRE SUA DÚVIDA
O que você quer saber sobre o seu trabalho?
Pergunte do seu jeito. A resposta vem em linguagem simples e com a fonte consultada.
Ex.: Quantos dias de férias eu tenho direito?
0/500
Perguntar
TEMAS MAIS PROCURADOS
Férias
2 perguntas
13º salário
2 perguntas



Consulta no celular

Cadastro no celular


Oráculo
DIREITOS TRABALHISTAS
Entrar
RECEBA POR E-MAIL
Crie sua conta
Com a conta você escolhe temas para acompanhar e recebe as novidades deles no seu e-mail.
Nome
E-mail
nome@exemplo.com
É para este e-mail que enviaremos as novidades.
Senha



Cadastro no celular

A resposta gerada pela IA (no painel azul, com o convite "Quer saber mais sobre o assunto? Sim") ainda não tem print com dados reais: depende da chave gratuita do Gemini no `backend/.env`.

Ajustes desta Sprint

Commit	O que mudou
`10f67db`	Busca na base, resposta com IA e "Saiba mais"
`231dc75`	IA trocada pelo Google Gemini gratuito (projeto sem custo)
`7f76733`	"Saiba mais" integrado à conta
`04f09ab`	Integração "Incluir na base" a partir das perguntas sem resposta; menu por perfil
`ad724c3`	Correção do redirecionamento depois de sair do sistema
`d916d51`	Convite "Quer saber mais sobre o assunto? Sim" com explicação antes do cadastro

Mudanças em relação ao planejado

O histórico completo das alternativas testadas está no registro de decisões.

- **IA:** a primeira versão usava o Claude (Anthropic), que é pago; foi trocado pelo **Google Gemini** no plano gratuito, para atender ao RNF09 (custo operacional).

- **Links e vídeos:** são escolhidos e conferidos pela equipe, por assunto (tabela `materiais_complementares`), sem pesquisa automática; assim os endereços são sempre reais e o envio não depende de cota de API. A primeira versão tem férias e 13º salário.
- **Conteúdo complementar:** enviado **por e-mail** pelo Gmail, de graça, sem exigir conta: a pessoa informa só o nome e o e-mail (decisão da equipe, por ser o caminho mais simples para quem usa e para quem mantém o sistema).

Sprint 07 — Dashboard, relatórios, filtros, exportação e histórico

Situação: Concluída e testada com o banco real

O que a Sprint pediu

Dashboard com dados reais, relatórios, pesquisas, filtros, exportação (quando aplicável) e logs ou histórico de operações.

O que foi entregue

Item pedido	Entrega	Onde ver
Dashboard	Indicadores (perguntas, taxa de resposta, pedidos de Saber mais, pessoas acompanhando temas), gráfico de perguntas por data, temas mais consultados, temas acompanhados, perguntas sem resposta e usuários por perfil	Tela **Dashboard** · `rotas/relatorios.py`
Relatórios	Relatório de perguntas e relatório do histórico de operações	Telas **Relatório de perguntas** e **Histórico de operações**
Pesquisas	Pesquisa no texto das perguntas, dos trechos da base e do histórico (pessoa ou descrição)	Campos "Pesquisar" de cada tela
Filtros	Período (de/até ou 7, 30, 90 dias e 12 meses), resultado, tema, ação e área	Barra de filtros de cada tela
Exportação	**CSV** com separador `;` e acentuação correta, que abre direto no Excel	Botão **Exportar CSV**
Histórico de operações	Logins, cadastros, inclusões, alterações, exclusões, publicações e pedidos de conteúdo por e-mail, com quem fez e quando	`supabase/005_historico_de_operacoes.sql` · `historico.py`

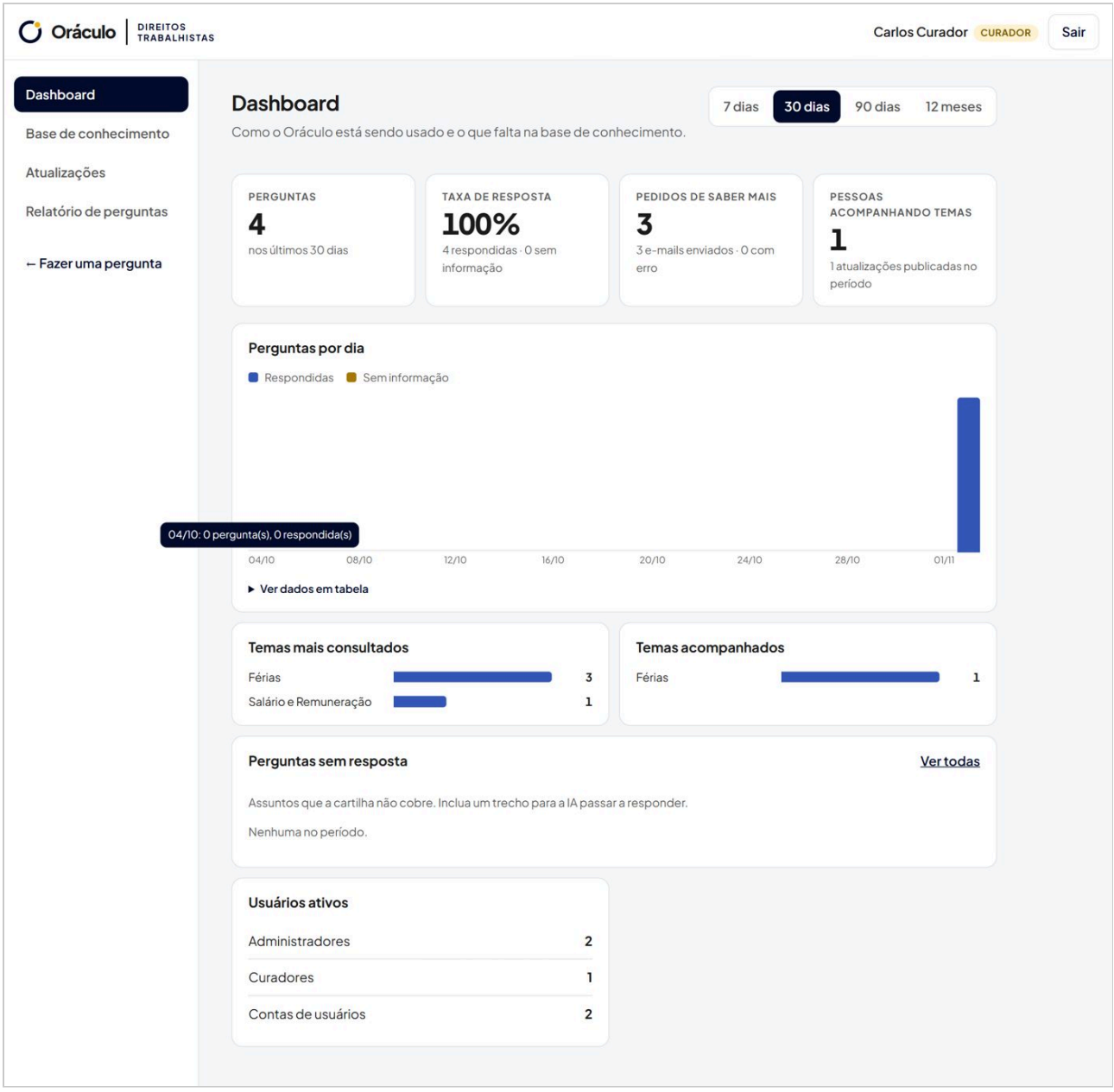
Decisões

- **Exportação em CSV, e não em PDF:** os relatórios são tabelas que a equipe tende a analisar e filtrar em planilhas; o CSV abre no Excel e no Google Planilhas sem perder dados.
- **Histórico só para o administrador:** ele mostra ações de todas as pessoas; o curador vê o dashboard e o relatório de perguntas, que não têm dados pessoais.
- **Falha no histórico não bloqueia a operação:** se o registro falhar, a ação do usuário continua e o erro vai para o log do servidor.
- **Gráficos:** cores validadas para contraste e daltonismo, dica ao passar o mouse e a opção "Ver dados em tabela" para acessibilidade.

Evidências

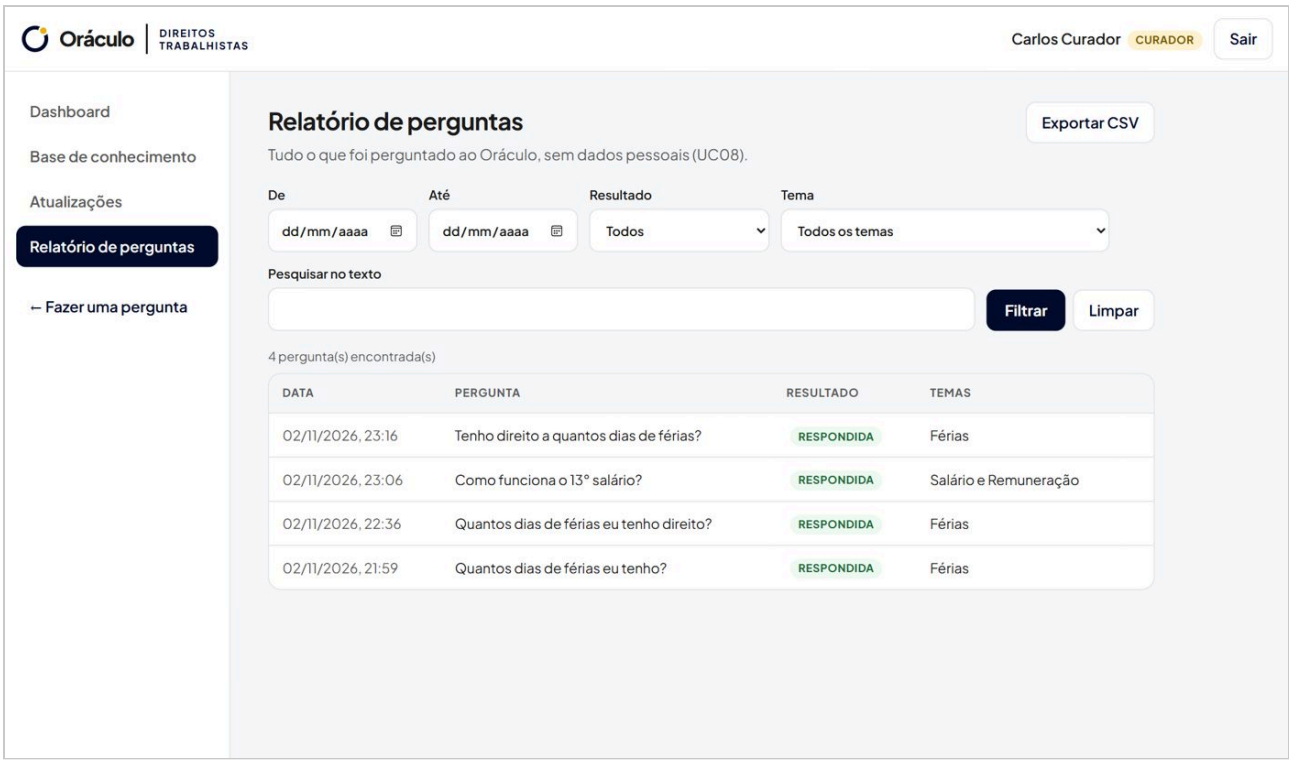
Prints gerados em 02/11/2026 pelo roteiro automatizado `capturar_evidencias.py`, com o sistema rodando localmente e o banco real no Supabase. Todos os prints estão em `docs/evidencias/`.

Dashboard com filtro de período e dados reais



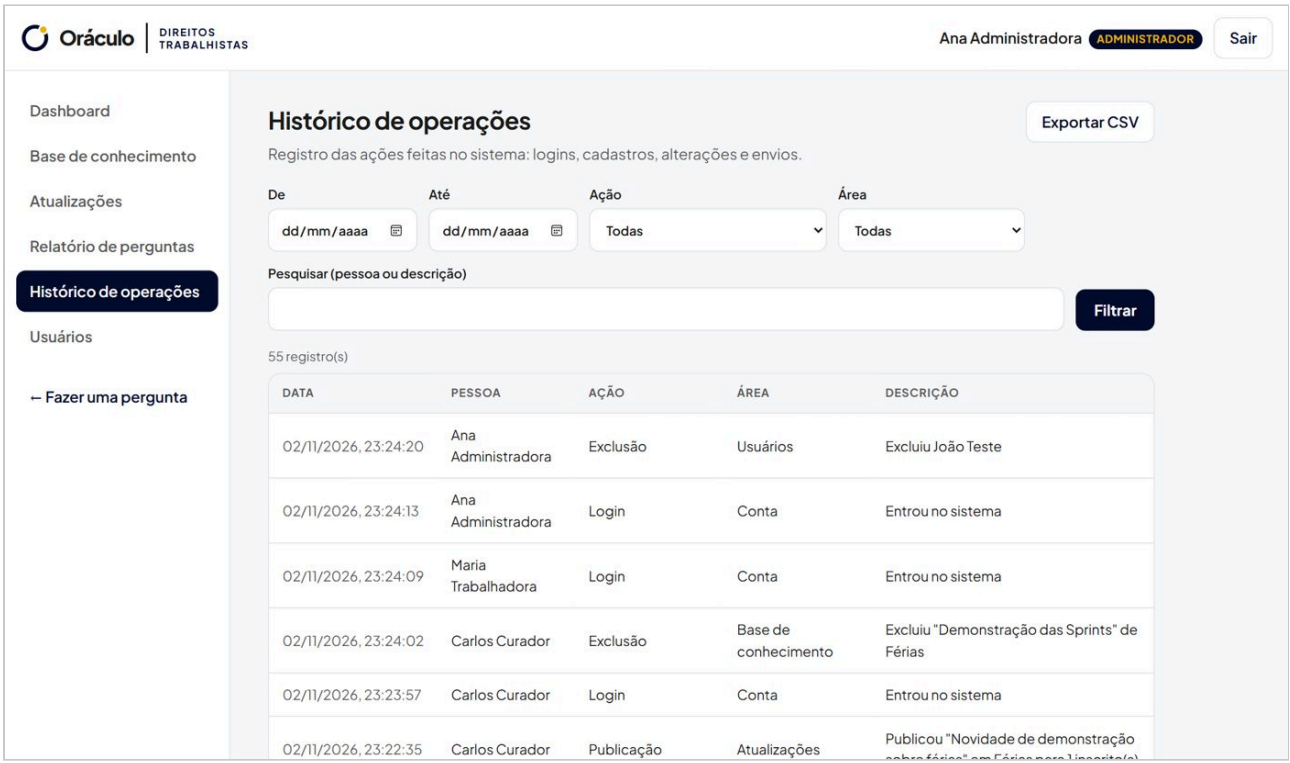
Dashboard com filtro de período e dados reais

Relatório de perguntas com filtros e exportação



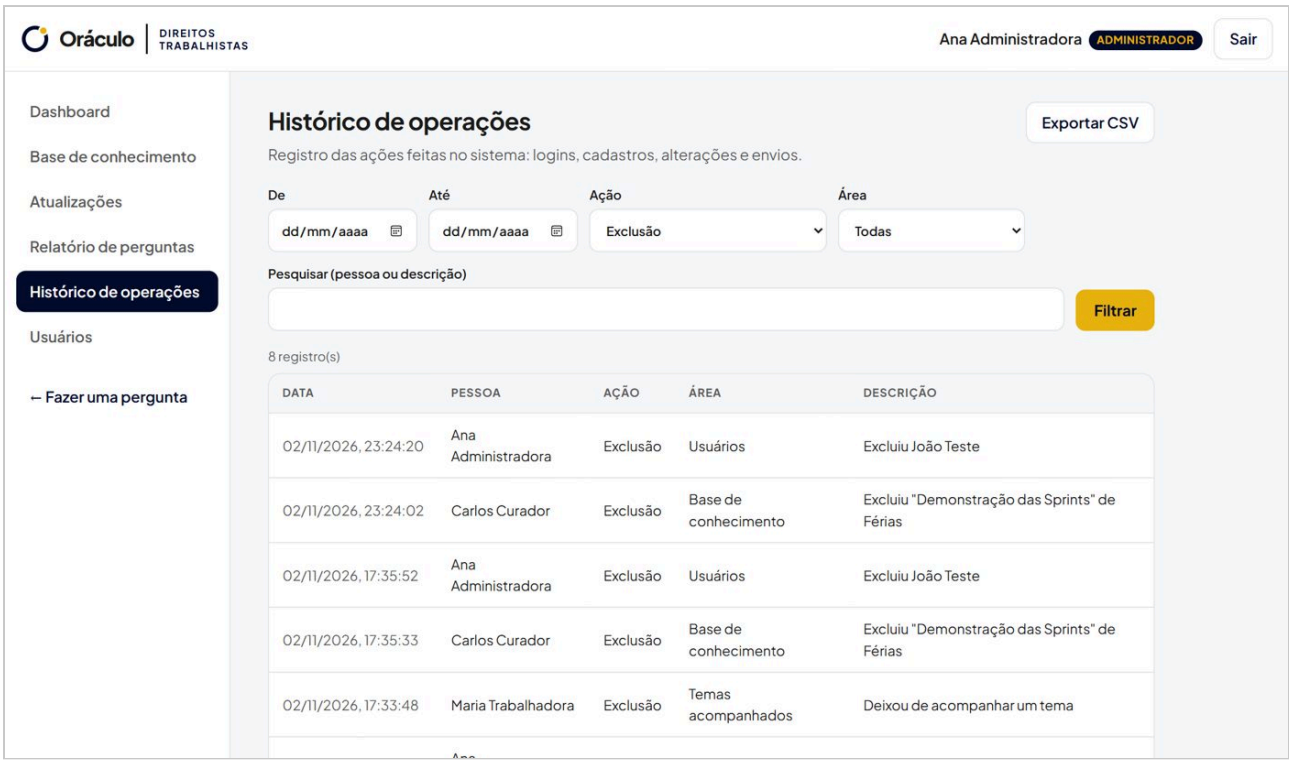
Relatório de perguntas com filtros e exportação

Histórico de operações



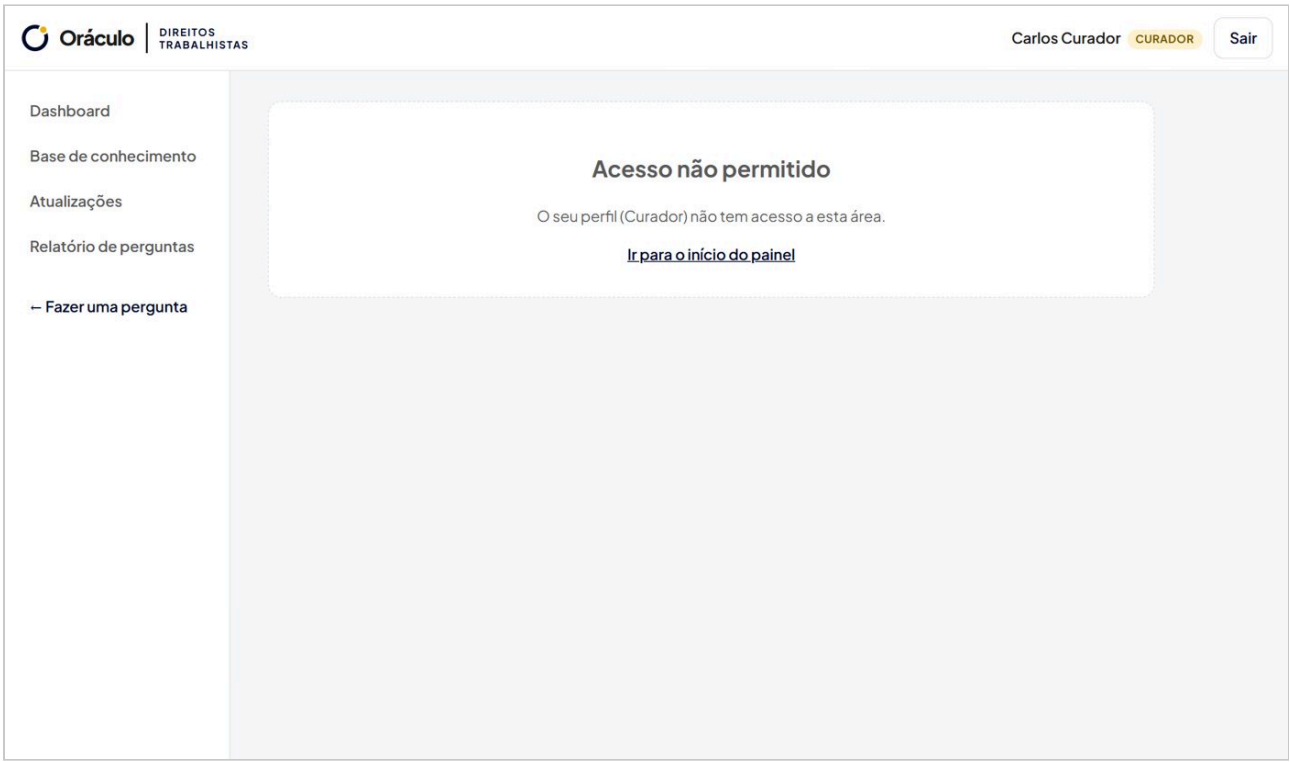
Histórico de operações

Histórico filtrado pela ação "Exclusão"



Histórico filtrado pela ação "Exclusão"

Histórico restrito ao administrador



Histórico restrito ao administrador

Arquivos exportados no teste: `historico-de-operacoes.csv` e `relatorio-perguntas.csv` (este só com o cabeçalho, porque sem a chave do Gemini nenhuma pergunta foi respondida ainda).

Ajustes desta Sprint

Commit	O que mudou
`57ad47c`	Backend: indicadores, relatórios com filtros, CSV e histórico de operações
`04f09ab`	Telas: Dashboard, relatórios, histórico e exportação

Sprint 08 — Consolidação: permissões, usabilidade e regras de negócio

Situação: Em andamento

O que a Sprint pediu

Todas as funcionalidades previstas implementadas, controle de permissões revisado, melhorias de usabilidade, revisão geral das regras de negócio e registro das mudanças de escopo.

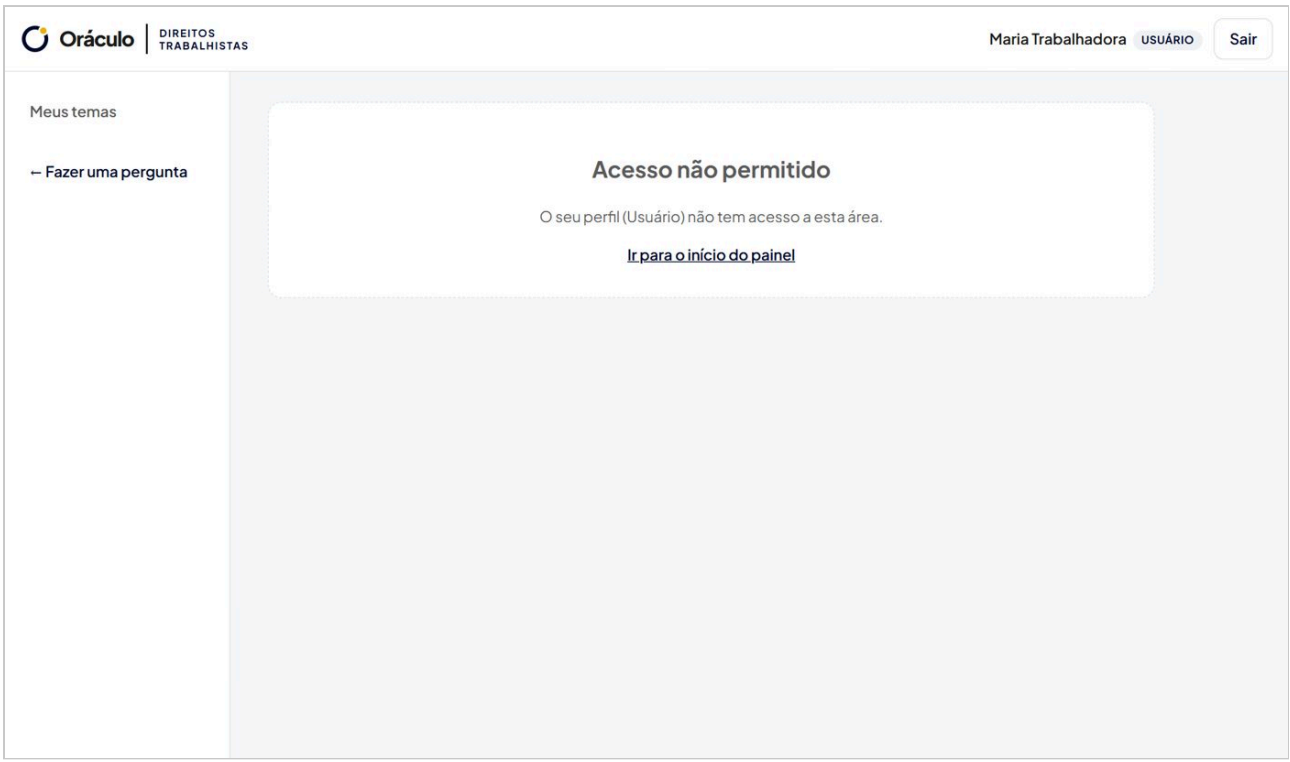
Controle de permissões (situação atual)

Funcionalidade	Visitante	Usuário	Curador	Administrador
Fazer perguntas e ver respostas	Sim	Sim	Sim	Sim
Receber "Saber mais" por e-mail	Sim	Sim	Sim	Sim
Acompanhar temas e ver novidades	—	Sim	—	—
Dashboard e relatório de perguntas	—	—	Sim	Sim
Base de conhecimento (CRUD)	—	—	Sim	Sim
Sinônimos da busca e sites e vídeos do Saber mais	—	—	Sim	Sim
Publicar atualizações	—	—	Sim	Sim
Excluir atualizações	—	—	Só as próprias	Todas
Histórico de operações	—	—	—	Sim
Gestão de usuários e perfis	—	—	—	Sim

As permissões são verificadas **no servidor**, em cada rota: o menu só esconde o que a pessoa não pode usar, mas o bloqueio real acontece na API (erro 401 sem login e 403 sem permissão). Os testes automatizados cobrem acessos permitidos e negados para cada perfil.

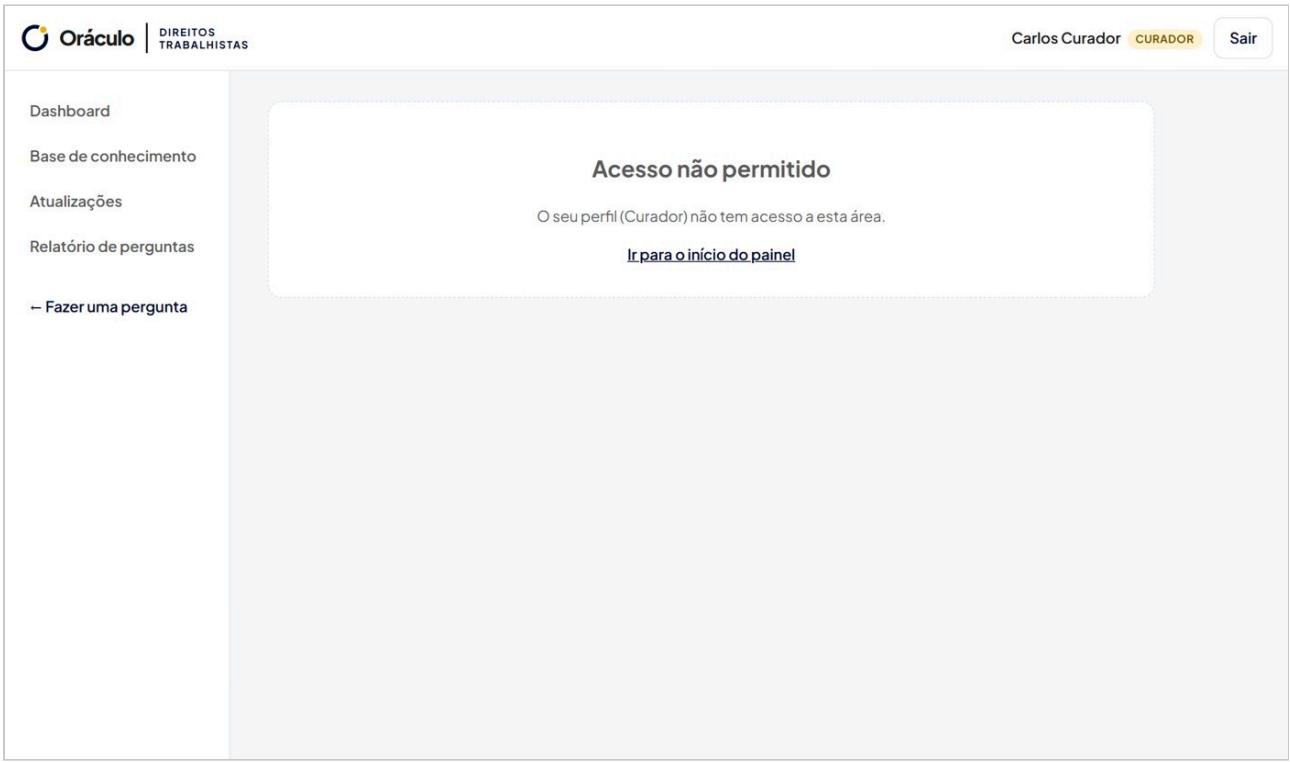
Evidências de permissões

Usuário comum tentando abrir a base de conhecimento



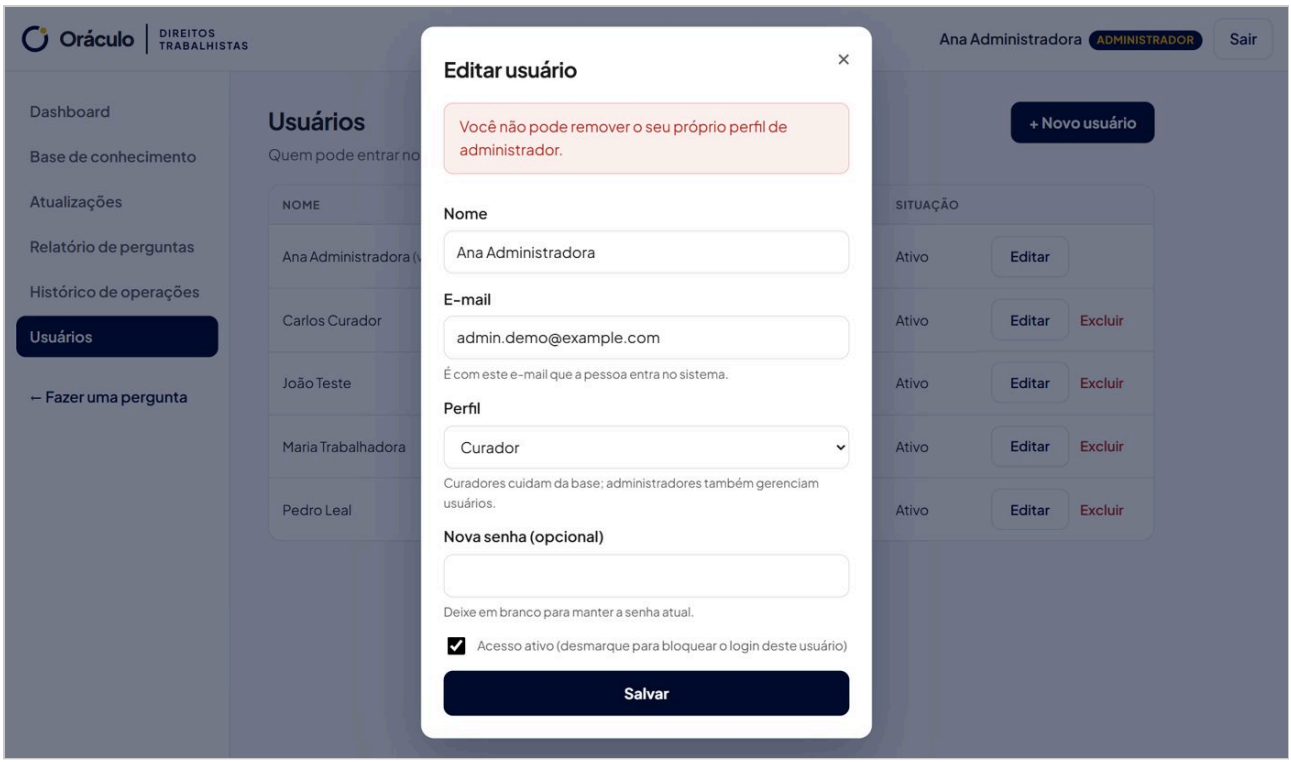
Usuário comum tentando abrir a base de conhecimento

Curador tentando abrir o histórico (exclusivo do administrador)



Curador tentando abrir o histórico (exclusivo do administrador)

Regra: o sistema não pode ficar sem administrador ativo



Regra: o sistema não pode ficar sem administrador ativo

Requisitos da Sprint 01

Requisito	Situação
RF01 a RF03 (consulta sem conta, sugestões, pergunta livre)	Implementados
RF04 a RF09 (RAG, resposta simples, fontes, conteúdo original, sem informação)	Implementados e testados com a IA real; avaliação das respostas em avaliacao-ia.md
RF10 e RF11 (conteúdo complementar por e-mail)	Implementados e testados com envio real pelo Gmail
RF12 e RF13 (acompanhar temas e receber atualizações)	Implementados (Módulo 2)
RF14 (área administrativa)	Implementado (painel com perfis)

Melhorias de usabilidade (revisão da equipe, 03/11/2026)

Antes	Depois	Por quê
Botão **Entrar** no topo da tela de consulta	Removido da consulta; a equipe entra pelo endereço `/entrar`	Quem consulta não precisa de conta: pergunta e, se quiser, usa o "Saber mais"
Cada fonte abria o mesmo texto que já estava na resposta	Cada seção da cartilha aparece uma vez, só com o link **Ver na cartilha**	Evita repetição e leva direto ao conteúdo original
O botão **Receber por e-mail** ficava ativo com o formulário vazio	Só fica disponível com nome, e-mail válido e a autorização da LGPD marcada, com um aviso do que falta	Deixa claro o que precisa ser preenchido e reforça o consentimento (RNF03)

Mudanças de escopo

O que foi pensado, testado e trocado (IA, WhatsApp pela Twilio, pela API da Meta e pelo n8n, pesquisa automática de links e publicação), com o motivo de cada decisão, está no registro de decisões.

Pendências

- **Conversa com o outro orientador sobre paralelismo:** pedida pela orientadora para confirmar que o conceito está aplicado. O material para a conversa está em paralelismo.md.
- Revisão de usabilidade com o público-alvo e registro das melhorias.
- Revisão final das regras de negócio após os testes com os serviços reais.

Sprint 09 — Verificação completa e documentação

Situação: Concluída: testes com o banco real, a IA (Gemini) e o envio de e-mail reais; prints refeitos em 03/11/2026

O que a Sprint pediu

Testes funcionais, correção de bugs, testes de validação, testes de navegação e atualização da documentação técnica.

Como os testes foram feitos

Tipo	Ferramenta	Resultado
Automatizados (backend)	103 testes com `pytest` (rodar `pytest` em `backend/`), com versões simuladas do banco, da IA e do e-mail	103 de 103 passando
Funcionais, de validação e de navegação	Roteiro `capturar_evidencias.py`, que usa o navegador Edge, o sistema rodando localmente e o **banco real** no Supabase. Cada passo verifica o que deve aparecer na tela e salva um print	35 de 35 passos aprovados (03/11/2026)
Integração contínua	GitHub Actions: a cada envio ao GitHub, roda os testes do backend e confere e compila o frontend	Automático a cada envio
Paralelismo	`test_paralelo.py`: uma barreira só libera quando as tarefas rodam juntas; envio das novidades e dashboard em paralelo (documento)	Dashboard 2,3 vezes mais rápido no banco real
Avaliação das respostas da IA	`avaliar_respostas.py`: 15 perguntas de referência com a IA real	Ver avaliação
Persistência	O roteiro roda em duas fases; entre elas o sistema é encerrado e aberto de novo	Dados mantidos

Casos de teste (sistema real, 02 e 03/11/2026)

#	Caso de teste	Perfil	Resultado esperado	Resultado obtido	Evidência
CT01	Entrar sem preencher os campos	Visitante	Mensagem "Preencha o campo e-mail."	Conforme esperado	print
CT02	Entrar com senha errada	Visitante	Mensagem genérica, sem revelar se a conta existe	Conforme esperado	print
CT03	Cadastro com nome curto, e-mail e senha inválidos e senhas diferentes	Visitante	Uma mensagem junto de cada campo	Conforme esperado	print
CT04	Cadastro com e-mail já usado	Visitante	"Já existe um usuário com este e-mail."	Conforme esperado	print

#	Caso de teste	Perfil	Resultado esperado	Resultado obtido	Evidência
CT05	Cadastro válido	Visitante	Conta criada e pessoa levada a "Meus temas"	Conforme esperado	print
CT06	Acompanhar tema sem escolher o tema	Usuário	"Preencha o campo tema."	Conforme esperado	print
CT07	Acompanhar o tema Férias	Usuário	Confirmação e tema na lista	Conforme esperado	print
CT08	Abrir a base de conhecimento pelo endereço	Usuário	"Acesso não permitido"	Conforme esperado	print
CT09	Entrar como curador	Curador	Abre o Dashboard com dados reais	Conforme esperado (após a correção do bug B06)	print
CT10	Filtrar a base por tema e pesquisar "recesso"	Curador	Só trechos do tema com o termo	Conforme esperado	print
CT11	Salvar trecho sem campos	Curador	"Preencha o campo tema." e demais campos	Conforme esperado	print
CT12	Cadastrar, editar e salvar um trecho	Curador	Mensagens "Trecho cadastrado." e "Trecho atualizado."	Conforme esperado	print
CT13	Publicar atualização sem título	Curador	"Preencha o campo título."	Conforme esperado	print
CT14	Publicar atualização do tema Férias sem o e-mail configurado	Curador	Publicação salva; envio marcado com erro e botão "Reenviar"	Conforme esperado	print
CT15	Abrir o histórico de operações	Curador	"Acesso não permitido" (exclusivo do administrador)	Conforme esperado	print
CT16	Fechar e abrir o sistema e procurar o trecho cadastrado	Curador	Trecho continua salvo	Conforme esperado	print
CT17	Excluir o trecho	Curador	Confirmação e depois "Trecho excluído."	Conforme esperado	print
CT18	Exportar o relatório de perguntas	Curador	Arquivo CSV baixado	Conforme esperado	CSV
CT19	Ver as novidades do tema acompanhado	Usuário	A atualização publicada aparece em "Meus temas"	Conforme esperado	print
CT20	Salvar novo usuário sem dados	Administrador	"Preencha o campo nome."	Conforme esperado	print
CT21	Rebaixar o próprio perfil de administrador	Administrador	Bloqueado com explicação	Conforme esperado	print
CT22	Excluir o usuário cadastrado no CT05	Administrador	Confirmação e "Usuário excluído."	Conforme esperado	print
CT23	Filtrar o histórico pela ação "Exclusão" e exportar	Administrador	Só exclusões, com quem fez e quando; CSV baixado	Conforme esperado	print · CSV

#	Caso de teste	Perfil	Resultado esperado	Resultado obtido	Evidência
CT24	Consulta, cadastro e painel no celular	Todos	Telas adaptadas, sem rolagem lateral	Conforme esperado	print
CT25	Perguntar "Quantos dias de férias eu tenho direito?" com a IA real	Visitante	Resposta da cartilha em cerca de 15 s, com fontes e o convite "Quer saber mais?"	Conforme esperado (03/11/2026)	—
CT26	"Saber mais" com nome e e-mail reais	Visitante	E-mail recebido com a resposta, 2 sites (Jusbrasil e cartilha do TRT4) e 2 vídeos (CEISC e Contas Online); pedido marcado como "enviado" no banco	Conforme esperado (03/11/2026)	—
CT27	Perguntar "Qual é a capital da França?" e uma pergunta sobre imposto de renda de empresas no exterior	Visitante	Aviso de que a cartilha não trata do assunto, sem fontes e sem o convite "Quer saber mais?" (RF09)	Conforme esperado (04/11/2026)	—
CT28	Cadastrar e excluir um sinônimo e um link do Saber mais no banco real	Curador	Inclusão e exclusão salvas, com registro no histórico	Conforme esperado (04/11/2026)	print
CT29	Salvar um link sem preencher os campos	Curador	Uma mensagem junto de cada campo obrigatório	Conforme esperado (04/11/2026)	print

Bugs encontrados e corrigidos

#	Bug	Como foi encontrado	Correção
B01	A busca não encontrava "estágio" nem com a palavra exata	Teste com o banco real	Função de busca corrigida (SQL 002)
B02	"Demitido", "patrão" e "FGTS" não encontravam os temas certos	Teste com o banco real	Tabela de sinônimos
B03	Erro 500 intermitente ao abrir a base	Roteiro de navegação	Repetição automática quando a conexão cai (`4165443`)
B04	Filtro de período do dashboard sempre recusado	Testes automatizados	Validação própria do período
B05	A dica do gráfico cobria a legenda	Revisão dos prints	Dica posicionada sobre a barra (`d916d51`)
B06	Depois de sair, a próxima pessoa que entrava era levada para a página da pessoa anterior	Roteiro de navegação	O endereço de retorno não é guardado quando a pessoa clica em Sair (`ad724c3`)
B07	A busca deixava o trecho sobre diaristas em 32º lugar para "Diarista tem carteira assinada?", e a IA respondia que a cartilha não tratava do assunto	Avaliação das respostas da IA	Reordenação com mais peso para as palavras raras (`ranqueamento.py`); o trecho subiu para o 1º lugar

Pendências e como serão resolvidas

Pendência	Ação prevista
Mais assuntos com sites e vídeos escolhidos	A equipe envia os links das demais perguntas sugeridas

Paralelismo e concorrência no Oráculo

Documento preparado para a conversa com o outro orientador, a pedido da orientadora, para confirmar se o conceito de paralelismo está aplicado no software. Na visão da equipe, **sim**: em várias camadas, com código, testes automatizados e uma medição de ganho no sistema real.

> **Situação:** conversa com o orientador **pendente**. As observações dele serão registradas aqui.

Conceitos usados

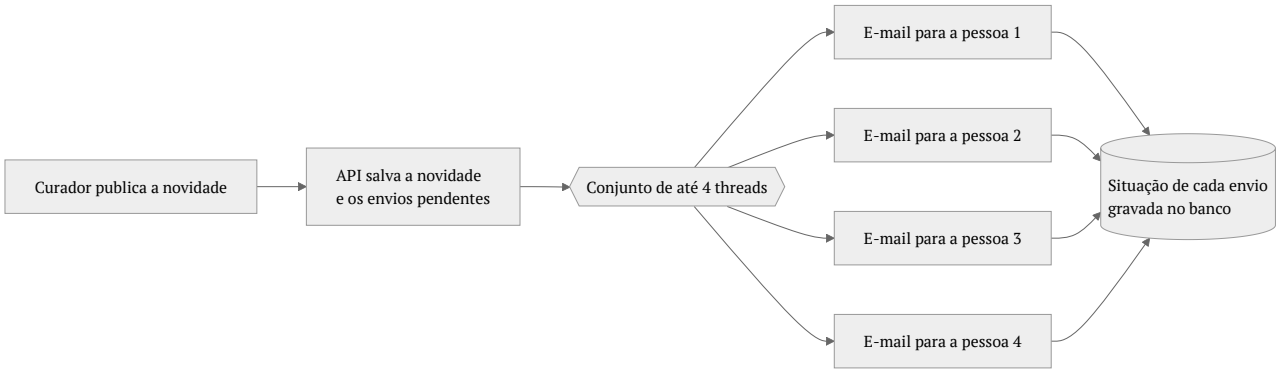
- **Concorrência:** o sistema lida com várias tarefas no mesmo período, alternando entre elas (por exemplo, responder à tela enquanto um e-mail é enviado).
- **Paralelismo:** várias tarefas executam ao mesmo tempo, em threads ou máquinas diferentes.
- **Tarefas do Oráculo:** quase todas passam o tempo **esperando a rede** (banco de dados no Supabase, IA do Google Gemini, Gmail). Para esse tipo de tarefa, threads são a escolha adequada em Python: enquanto uma thread espera a resposta da rede, o interpretador libera as outras (o GIL é liberado durante a espera), e as esperas acontecem ao mesmo tempo. Processos separados só seriam necessários para cálculo pesado de CPU, que o sistema não tem.

Onde o paralelismo está aplicado

Camada	O que roda em paralelo ou de forma concorrente	Como	Onde está no código
Servidor web	Pedidos de várias pessoas ao mesmo tempo	O FastAPI executa cada rota em um conjunto de threads (até 40 por processo), então uma consulta lenta à IA não bloqueia as outras pessoas	Rotas em `backend/app/rotas/` (funções `def`, executadas pelo FastAPI em threads)
Envio das novidades por e-mail	Os e-mails para cada pessoa que acompanha o tema	`ThreadPoolExecutor` com até 4 envios simultâneos; a falha de um envio não atrasa nem impede os outros	`backend/app/rotas/atualizacoes.py` e `backend/app/paralelo.py`
Dashboard	As 5 consultas ao banco que montam os indicadores (perguntas, pedidos, inscrições, usuários, atualizações)	As consultas são independentes e vão ao banco ao mesmo tempo	`backend/app/rotas/relatorios.py`

Camada	O que roda em paralelo ou de forma concorrente	Como	Onde está no código
"Saber mais" em segundo plano	O envio do e-mail e a tela	A tela recebe a confirmação na hora e acompanha a situação do pedido enquanto o servidor monta e envia o e-mail (tarefa em segundo plano)	`backend/app/segundo_plano.py` e `backend/app/rotas/consulta.py`
Interface (navegador)	Requisições à API	A tela de sites e vídeos busca os links e os temas ao mesmo tempo (`Promise.all`)	`frontend/src/paginas/painel/Materiais.jsx`
Publicação (Vercel)	Instâncias do backend	A Vercel cria várias instâncias da função conforme o número de acessos (escala horizontal)	`vercel.json`
Banco de dados	Consultas de vários usuários	O PostgreSQL (Supabase) atende conexões simultâneas, com controle de concorrência próprio	`supabase/`
Integração contínua	Verificação do backend e do frontend	O GitHub Actions roda as duas verificações em máquinas separadas, ao mesmo tempo	`.github/workflows/testes.yml`

Exemplo: envio das novidades de um tema



O limite de 4 envios simultâneos é proposital: mais conexões ao mesmo tempo poderiam ser recusadas pelo Gmail.

Medição no sistema real

As 5 consultas do dashboard, feitas ao banco real no Supabase (mediana de 5 execuções, 06/11/2026):

Forma de execução	Tempo
Uma depois da outra	0,99 s
Em paralelo	0,43 s (**cerca de 2,3 vezes mais rápido**)

Como os testes comprovam

Os testes em `backend/tests/test_paralelo.py` usam uma **barreira** (`threading.Barrier`): ela só libera quando todas as tarefas estão rodando juntas. Se as tarefas fossem executadas uma de cada vez, a barreira nunca liberaria e o teste falharia por tempo. Os testes cobrem:

- a função que executa tarefas em paralelo devolve os resultados na ordem certa;
- as tarefas de fato rodam ao mesmo tempo;
- os e-mails de uma novidade são enviados aos inscritos em paralelo e cada envio fica registrado como "enviado".

Esses testes rodam automaticamente a cada envio ao GitHub.

Cuidados tomados

- **Tarefas independentes:** cada thread trabalha com dados próprios (um e-mail, uma consulta), sem disputar a mesma variável.
- **Limite de threads:** o número de tarefas simultâneas é limitado, para não sobrecarregar o banco nem o Gmail.
- **Falhas isoladas:** um erro em uma thread fica registrado no log e na situação daquele envio, sem derrubar as demais.
- **Plano gratuito da IA:** as perguntas ao Gemini continuam uma de cada vez por pessoa, porque o plano gratuito limita o número de pedidos por minuto e por dia.

Avaliação das respostas da IA

Data: 06/11/2026 · **Modelo:** Google Gemini (plano gratuito) · **Resultado:** 15 de 15 perguntas avaliadas aprovadas em todos os critérios

Avaliação do item PB12 do backlog e do RNF08: perguntas de referência feitas ao sistema real (banco no Supabase e IA), sem gravar nada no banco. Gerada por `docs/ferramentas/avaliar_respostas.py`.

Critérios

Critério	O que confere	Requisito
Respondeu quando devia	Responde as perguntas que a cartilha trata e recusa as que ela não trata	RF09
Fonte do tema certo	Pelo menos uma fonte citada é do tema esperado da cartilha	RF07, RNF08
Sem fontes inventadas	Todo trecho citado pela IA veio da busca no banco	RNF08
Resposta curta	Até 1200 caracteres e até 3 parágrafos	RF06

Resultado por pergunta

Pergunta	Tema esperado	Respondeu?	Temas das fontes	Critérios
Quantos dias de férias eu tenho direito?	Férias	Sim	Férias	Aprovada
Posso vender parte das minhas férias?	Férias	Sim	Férias	Aprovada
Quando o 13º salário deve ser pago?	Salário e Remuneração	Sim	Salário e Remuneração	Aprovada
Quem trabalhou só alguns meses recebe 13º?	Rescisão (término), Salário e Remuneração	Sim	Salário e Remuneração	Aprovada
Qual é a jornada máxima de trabalho por dia?	Jornada de Trabalho	Sim	Jornada de Trabalho, Salário e Remuneração	Aprovada
Como funciona o pagamento de horas extras?	Jornada de Trabalho, Salário e Remuneração	Sim	Jornada de Trabalho, Salário e Remuneração	Aprovada
O que recebo se for demitido sem justa causa?	Rescisão (término)	Sim	Rescisão (término)	Aprovada
Quanto tempo de aviso prévio eu preciso cumprir?	Rescisão (término)	Sim	Rescisão (término)	Aprovada
Estagiário tem direito a férias?	Contrato de Estágio	Sim	Contrato de Estágio	Aprovada
Qual é a carga horária máxima de um estágio?	Contrato de Estágio	Sim	Contrato de Estágio	Aprovada
Empregada doméstica tem direito a FGTS?	Contratações e modalidades, Fundo de Garantia, Relação de Emprego	Sim	Fundo de Garantia	Aprovada

Pergunta	Tema esperado	Respondeu?	Temas das fontes	Critérios
Diarista tem carteira assinada?	Contratações e modalidades, Relação de Emprego, Salário e Remuneração	Sim	Salário e Remuneração	Aprovada
Qual é a capital da França?	Fora da cartilha	Não	—	Aprovada
Qual é a melhor receita de bolo de cenoura?	Fora da cartilha	Não	—	Aprovada
Como declarar o imposto de renda de uma empresa no exterior?	Fora da cartilha	Não	—	Aprovada

Observações

- O plano gratuito do Gemini permite 20 pedidos por dia no modelo principal. Quando ele está sobrecarregado ou sem cota, o sistema usa o modelo reserva (com cota própria), e cada resposta pode levar de 10 a 25 segundos (RNF05).
- O sistema também descarta, antes de mostrar a resposta, qualquer trecho citado que não tenha vindo da busca.
- Perguntas fora da cartilha, como "Qual é a capital da França?", também foram testadas no sistema real no caso CT27 da Sprint 09.
- **Melhoria feita a partir da avaliação:** na primeira rodada, "Diarista tem carteira assinada?" falhou. A cartilha responde (seção Tipos de salário: até 2 dias por semana para a mesma família é diarista), mas a busca colocava esse trecho só na 32ª posição, porque palavras comuns ("carteira", "assinada") pesavam mais que a palavra rara "diarista". A busca passou a dar mais peso às palavras raras da cartilha (`backend/app/ranqueamento.py`): o trecho certo subiu para o 1º lugar, "Estagiário tem direito a férias?" subiu do 4º para o 1º, e a pergunta passou na nova rodada.

Registro de decisões e mudanças de escopo

Este documento reúne o que foi **pensado, testado e trocado** durante o desenvolvimento, com o motivo de cada escolha. Ele atende ao pedido da Sprint 08 de registrar as mudanças de escopo. O restante da documentação descreve só o que o projeto usa hoje; aqui fica o caminho até lá.

Cada decisão aponta o commit em que a mudança entrou, para quem quiser ver o código daquele momento.

Resumo

Assunto	O que foi considerado	O que ficou	Principal motivo
Inteligência artificial	Claude (Anthropic)	Google Gemini, plano gratuito	Custo zero (RNF09)
Envio do conteúdo complementar	Twilio, WhatsApp Cloud API (Meta), n8n com WAHA e com Evolution API	E-mail pelo Gmail	Entregas falhando, regras de uso, instalação complexa e risco de bloqueio do número
Conta de usuário	Cadastro e login pelo telefone (WhatsApp)	Cadastro e login pelo e-mail	Acompanhou a troca do WhatsApp pelo e-mail
Sites e vídeos do "Saber mais"	Pesquisa no Google pelo Gemini e outras buscas automáticas	Links escolhidos pela equipe	Cota indisponível no plano gratuito e endereços inventados pela IA
Publicação na internet	GitHub Pages, Render	Vercel	Rodar o backend em Python e permitir envio de e-mail, sem custo

1. Inteligência artificial: de Claude para Gemini

- **Primeira versão (31/10/2026, `10f67db`):** as respostas eram geradas pelo Claude, da Anthropic.
- **Problema:** a API do Claude é paga por uso, e o projeto tem como requisito não gerar custo (RNF09).
- **Decisão (31/10/2026, `231dc75`):** trocar pelo **Google Gemini**, que tem plano gratuito sem cartão de crédito. A forma de responder (só com os trechos da cartilha e com as fontes conferidas) continuou a mesma.
- **Ajuste posterior:** o modelo principal do plano gratuito às vezes fica sobrecarregado; nesse caso o sistema usa um modelo reserva do próprio Gemini (`e5b69e5`).

2. Envio do conteúdo complementar: do WhatsApp para o e-mail

A proposta da Sprint 01 previa enviar o "Saber mais" (resposta, 2 sites e 2 vídeos) e as novidades dos temas **pelo WhatsApp** (RF11, UC05, PB08). Foram testados quatro caminhos antes da troca para o e-mail.

2.1. Twilio (WhatsApp Sandbox)

- **Quando:** primeira versão, 31/10/2026 (`10f67db`).
- **Como funcionava:** a Twilio oferece um número de testes (Sandbox) que envia mensagens de WhatsApp por uma API.
- **Por que saiu:**
- cada pessoa precisava primeiro mandar um **código de entrada** ("join ...") para o número da Twilio antes de receber qualquer mensagem, o que não faz sentido para o público do Oráculo;
- depois do período de testes, o serviço é **pago** (RNF09).

2.2. WhatsApp Cloud API (API oficial da Meta)

- **Quando:** 02/11/2026 (`f26a29a`).
- **Como funcionava:** a API oficial do WhatsApp, com um número de testes gratuito da Meta e um token de acesso.
- **O que aconteceu no teste real:** a Meta **aceitava** o envio, mas a mensagem **não chegava** ao celular. O número de teste estava cadastrado no WhatsApp sem o 9 extra dos celulares brasileiros, e o sistema da Meta não fazia essa correspondência.
- **Outras limitações:** o número de teste só envia para até 5 números cadastrados manualmente; fora da janela de 24 horas só podem ser enviados modelos de mensagem aprovados pela Meta; o token de acesso de teste expira.
- **Incidente de segurança:** durante a configuração, um token de acesso da Meta e uma chave de API foram parar no arquivo de exemplo do repositório. Os valores foram retirados (`ec79687`, `4795de3`) e foi criado um teste automático que impede valores reais nos arquivos de exemplo.
- **Decisão da equipe:** não seguir com a API da Meta.

2.3. Automação no n8n com WAHA e com Evolution API

- **Quando:** 02/11/2026 (`ca6c3dd` e `2e4f094`).
- **Como funcionava:** o Oráculo avisava um fluxo do **n8n** (item PB09 do backlog, "Automação com n8n"), que enviava a mensagem por um conector de WhatsApp Web rodando no computador: primeiro o **WAHA**, depois a **Evolution API**, escolhida por ser muito usada no Brasil. O fluxo conferia como o número estava registrado no WhatsApp, o que resolvia o problema do 9 extra.
- **Por que saiu:**
- exigia instalar e manter o **Docker Desktop** (com WSL 2 no Windows), um banco PostgreSQL e o n8n, além de ler um QR code com o celular: complexo para a equipe e para quem for avaliar;
- só funcionava com o computador ligado, o que impediria publicar o sistema na internet;
- o WhatsApp Web **não é um serviço oficial** para automação e vai contra os termos de uso do WhatsApp, com risco de **bloqueio do número**.

2.4. Decisão: e-mail pelo Gmail

- **Quando:** 02/11/2026 (`e83e1ba`).

- **Como funciona:** a pessoa informa **nome e e-mail** (com a autorização da LGPD) e recebe a resposta, os sites e os vídeos por e-mail. O envio usa uma conta do Gmail com **senha de app**, sem custo e sem instalar nada.
- **O que mudou junto:** a conta de usuário (Módulo 2) passou a usar e-mail e senha no lugar do telefone, e as novidades dos temas acompanhados também chegam por e-mail. O banco foi ajustado pelo `006_contato_por_email.sql`.
- **Requisitos afetados:** RF11, UC05, PB08 e PB09 foram atualizados no relatório da Sprint 01.
- **Teste real:** aprovado em 03/11/2026 (CT26 da Sprint 09).

3. Sites e vídeos do "Saber mais": da pesquisa automática para links escolhidos

- **Primeira ideia:** o próprio Gemini pesquisaria no Google 2 sites e 2 vídeos do YouTube sobre o assunto, e o sistema só aceitaria endereços que viessem da pesquisa real.
- **O que aconteceu no teste real:** a resposta do Gemini funcionou, mas a **pesquisa no Google não está liberada** no plano gratuito da chave usada (erro de cota).
- **Alternativas testadas e descartadas:**
- pedir ao Gemini que sugerisse páginas oficiais sem pesquisar: das 6 sugeridas, **nenhuma existia** (endereços inventados pela IA);
- ler páginas de resultados de buscadores: um deles bloqueia acesso automático, e esse tipo de leitura quebra quando o site muda.
- **Decisão (02/11/2026, `24fee80`):** a equipe escolhe e confere os links de cada assunto, que ficam na tabela `materiais_complementares` (`007_materiais_complementares.sql`). Para assuntos ainda sem links, vão a página da cartilha do TRT4, o texto oficial da lei e uma busca pronta no YouTube.
- **Ganhos:** endereços sempre reais e conferidos, envio mais rápido e sem depender de cota de API.

4. Publicação na internet

Opção	Resultado
GitHub Pages (github.io)	Descartado: só hospeda páginas estáticas; não roda o backend em Python (IA, login, painel, e-mail) e não guarda chaves em segredo
Render (plano gratuito)	Descartado: o plano gratuito bloqueia o envio de e-mail, usado pelo "Saber mais"
Vercel (plano Hobby)	**Escolhido** (`45176c3`): gratuito, publica o site e o backend juntos a partir do GitHub, atualiza a cada envio e permite o e-mail. Guia em `publicacao.md`

Ajuste necessário: na Vercel o servidor "dorme" logo depois de responder, então lá o e-mail é enviado dentro do próprio pedido, em vez de depois da resposta.

5. Ajustes de usabilidade

As melhorias feitas depois da revisão da equipe com o sistema publicado (botão "Entrar" fora da consulta, fontes sem repetição e envio liberado só com o consentimento) estão na Sprint 08.

Histórico de alterações do relatório

Data	Alteração
31/10/2026	Seção 4 renomeada de "Identidade Visual" para "Objetivos", com a numeração 4.1, 4.2 e 4.3 alinhada ao conteúdo. A identidade visual passou a ter seção própria (14).
31/10/2026	Seções 6 e 9: incluída a coluna "Situação" em requisitos funcionais e backlog.
31/10/2026	Seção 7: observação atualizada, pois as tecnologias da aplicação foram definidas.
31/10/2026	Seção 10: Sprint 02 marcada como adiantada e Sprint 03 como iniciada.
31/10/2026	Seção 11: incluídas a estrutura do repositório, a estratégia de branches com Pull Requests e o registro de versões no CHANGELOG.
02/11/2026	Seção 11: o projeto passa a usar um único branch (`main`), com um commit por ajuste.
31/10/2026	Novas seções 12 (Tecnologias Definidas) e 13 (Andamento do Desenvolvimento).
31/10/2026	Nova seção 14 (Identidade Visual) e descrição do novo visual na seção 13.
31/10/2026	Seção 14: paleta alterada para azul-escuro `#030C2C` e dourado `#E8B20F`.
31/10/2026	Seções 6, 9, 11, 12 e 13: base de conhecimento no Supabase, respostas com IA e envio de conteúdo complementar.
31/10/2026	Seção 13: cartilha carregada no Supabase, busca validada com o banco real, tabela de sinônimos e limite de trechos por tema.
31/10/2026	Seção 12: a IA passou a ser o Google Gemini (plano gratuito), para manter o projeto sem custo (RNF09).
02/11/2026	Seções 5, 6, 8, 9, 12 e 13: o conteúdo complementar e as atualizações de temas passam a ser enviados por e-mail (RF11, UC05, PB08, PB09).
04/11/2026	Seções 6, 7, 9, 10, 12 e 13: situação dos requisitos e do backlog atualizada após os testes reais; integração contínua (GitHub Actions) e publicação (Vercel) incluídas nas tecnologias.
06/11/2026	Seção 1: Gerson Martinho Mesquita Neto (01604283) incluído entre os integrantes. Seção 12: paralelismo aplicado no envio das novidades e no dashboard (ver documento de paralelismo).
06/11/2026	Seção 13: a busca passou a dar mais peso às palavras raras da cartilha, melhoria encontrada na avaliação das respostas da IA (PB12).